

TXW81x 外设示例代码说明



保密等级	A	TXW81x 外设示例代码说明	文件编号	TX-0000
发行日期	2023-11-06		文件版本	V1.0
责任与版权 责任限制 由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。 客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。 在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。 版权申明 泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。 未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。				
		珠海泰芯半导体有限公司 Zhuhai Taixin Semiconductor Co.,Ltd	珠海市高新区港湾一号科创园港 11 栋 3 楼	
版权所有 侵权必究 Copyright © 2023 by Tai Xin All rights reserved				

保密等级	A	TXW81x 外设示例代码说明	文件编号	TX-0000																								
发行日期	2023-11-06		文件版本	V1.0																								
修订记录 <table><tr><th>日期</th><th>版本</th><th>描 述</th><th>修订人</th></tr><tr><td>2023-11-06</td><td>V1.0</td><td>初始版本</td><td>TX</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr></table>					日期	版本	描 述	修订人	2023-11-06	V1.0	初始版本	TX																
日期	版本	描 述	修订人																									
2023-11-06	V1.0	初始版本	TX																									
		珠海泰芯半导体有限公司 Zhuhai Taixin Semiconductor Co.,Ltd	珠海市高新区港湾一号科创园港 11 栋 3 楼																									
版权所有 侵权必究 Copyright © 2023 by Tai Xin All rights reserved																												

保密等级	A	TXW81x 外设示例代码说明	文件编号	TX-0000
发行日期	2023-11-06		文件版本	V1.0
目录 TXW81x 外设示例代码说明..... 1 1. 概述..... 1 2. 外设使用示例..... 5 2.1. UART..... 5 2.2. PWM..... 15 2.3. TIMER..... 20 2.4. I2C..... 25 2.5. GPIO..... 30 2.6. ADC..... 33 2.7. SPI..... 37 2.8. CAPTURE..... 41 2.9. SDHOST..... 46 2.10. PDM_AUDAC..... 49				
		珠海泰芯半导体有限公司 Zhuhai Taixin Semiconductor Co.,Ltd	珠海市高新区港湾一号科创园港 11 栋 3 楼	
版权所有 侵权必究 Copyright © 2023 by Tai Xin All rights reserved				

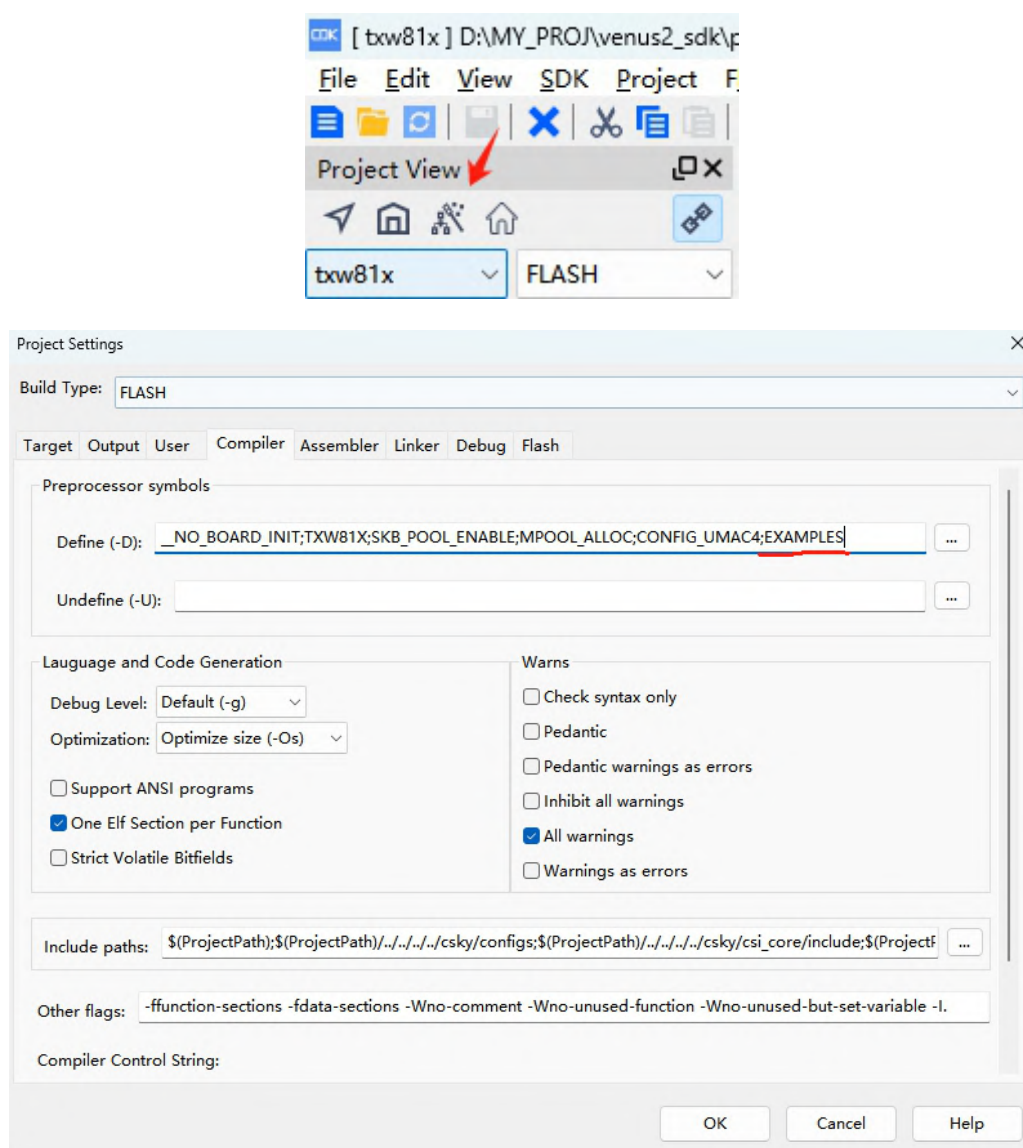
1. 概述

本文档介绍如何使用芯片外设实现某些功能。SDK 外设示例的主体框架介绍如下：

1) 外设示例代码的运行逻辑

若要运行外设示例代码，需要在工程设置中，定义 EXAMPLES 宏，如下：

点击魔术棒，进行相应的 Project Settings。



在定义 EXAMPLES 宏后，可依据运行的外设示例在 project_config.h 中，再定义该示例的宏。注意：每次只能运行单个外设示例，不能同时运行多个外设示例。

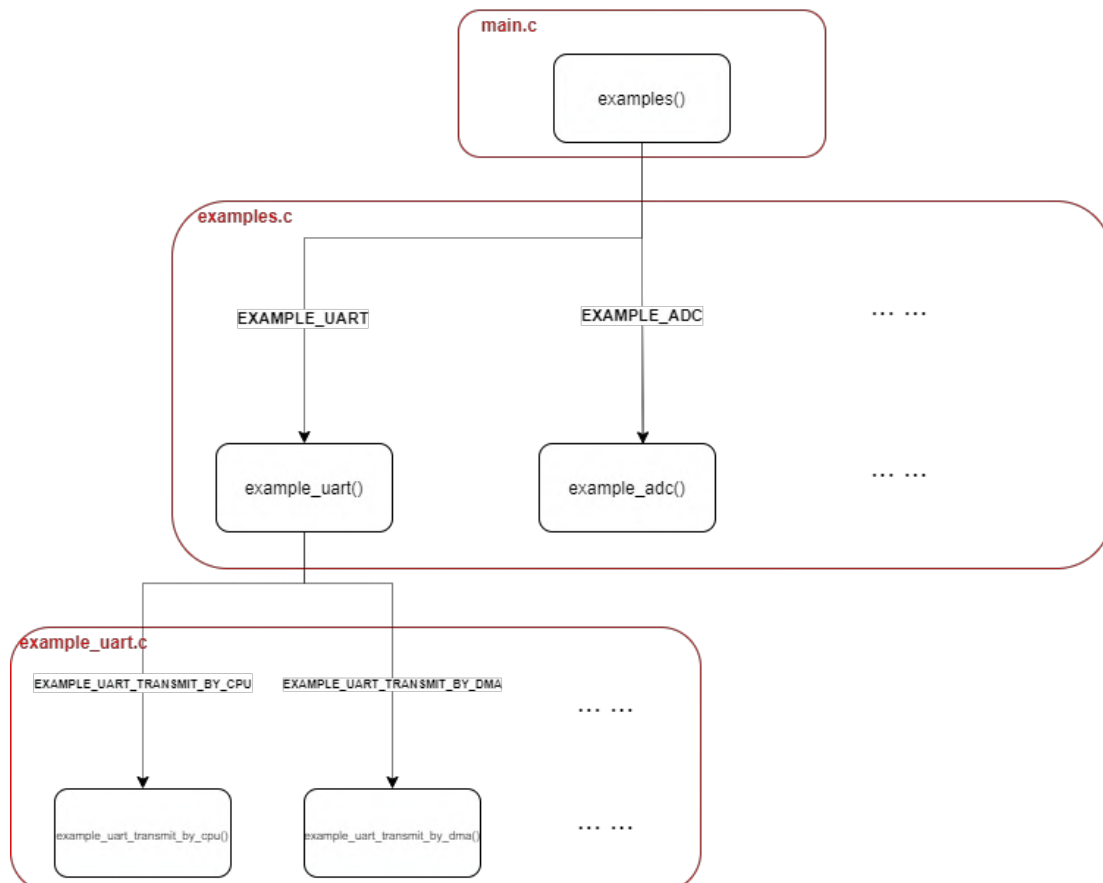
以 UART 外设为例，如下：

```

project_config.h X
46 #define PIN_SPI0_IO1 PB_10
47 #define PIN_SPI0_IO2 PB_11
48 #define PIN_SPI0_IO3 PB_12
49
50 #define PIN_PDM_MCLK PA_2
51 #define PIN_PDM_DATA PA_0
52
53
54 //每次只能开一个demo测试
55 #define EXAMPLE_UART
56 //#define EXAMPLE_ADC
57 //#define EXAMPLE_CAPTURE
58 //#define EXAMPLE_GPIO
59 //#define EXAMPLE_I2C
60 //#define EXAMPLE_SPI
61 //#define EXAMPLE_PDM_SDC
62 //#define EXAMPLE_PDM_AUDAC
63 //#define EXAMPLE_AUDIO
64 //#define EXAMPLE_PWM
65 //#define EXAMPLE_SDHOST
66 //#define EXAMPLE_TIMER
67 //#define EXAMPLE_RGB

```

SDK 中的外设示例代码框架如下（由宏进行控制）：



2) 外设示例代码内容框架（以UART示例）：

示例代码位于SDK目录：

... \project\examples\peripherals\uart

包含以下文件：

example_uart.c

example_uart.h

example_uart.c 文件中的 example_uart 函数，包含了 UART 的所有使用示例。

example_uart 函数中的每个使用示例，通过 example_uart.c 文件首部的宏进行控制。

（注意：只能运行一个使用示例，不能运行多个使用示例）



```
example_uart.c
332  * @brief  UART example main function.
333  * @param  none.
334  * @param  none.
335  * @retval none.
336  * @note
337  *      UART example main function.
338  *      You need to configure relevant macro that open the specifical exam
339  *      Particularly, Ensuring only one macro work is important.
340  */
341  void example_uart(void)
342  {
343
344  /**
345   * @brief  This example shows how to use CPU to send/get data.
346   * @param  none.
347   * @param  none.
348   * @retval none.
349   */
350   #if EXAMPLE_UART_TRANSMIT_BY_CPU
351       example_uart_transmit_by_cpu();
352   #endif
353
354
355  /**
356   * @brief  This example shows how to use DMA to send/get data.
357   * @param  none.
358   * @param  none.
359   * @retval none.
360   */
361   #if EXAMPLE_UART_TRANSMIT_BY_DMA
362       example_uart_transmit_by_dma();
363   #endif
364
365
366  /**
367   * @brief  This example shows how to use UART interrupt.
368   * @param  none.
369   * @param  none.
```

```
14  /*
15
16  /* Includes -----*/
17  #include "tx_platform.h"
18  #include "typesdef.h"
19  #include "list.h"
20  #include "dev.h"
21  #include "devid.h"
22  #include "osal/string.h"
23  #include "osal/senaphore.h"
24  #include "osal/autext.h"
25  #include "osal/asqueue.h"
26  #include "osal/irq.h"
27  #include "osal/task.h"
28  #include "osal/sleep.h"
29  #include "hal/uart.h"
30
31  #ifndef EXAMPLE_UART
32
33  /* Private macro -----*/
34
35  #define EXAMPLE_UART_TRANSMIT_BY_CPU          (0)
36
37  #define EXAMPLE_UART_TRANSMIT_BY_DMA         (0)
38
39  #define EXAMPLE_UART_USE_IRQ                 (0)
40
41  #define EXAMPLE_UART_USE_IOCTL_CMD           (0)
42
43
44
45
46  /**
47   * @brief This example shows how to configure UART module through IOCTL_CMD.
48   * @param none.
49   * @param none.
50   * @retval none.
51   */
52  #if EXAMPLE_UART_USE_IOCTL_CMD
53  int32 example_uart_use_ioctl_cmd(void)
54  {
55      struct uart_device *p_uart = NULL;
56      uint8 str[] = "TAIXIN-SEMI SDK, UART example test\r\n";
57      p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
58
59      if (!p_uart) {
60          return RET_ERR;
61      }
62
63      /*!
64       * open UART with 115200 baudrate.
65       */
66      uart_open(p_uart, 115200);
67
68      /*!
69       * configure UART baudrate, using "uart_ioctl" API.
70       */
71  }
```

3) 硬件连接公共部分说明

后续章节的外设示例中“硬件连接”的部分均要使用，如下：

a) 硬件资源

- TXW816_WAV_EVB开发板
- 5V电源
- 平头哥CKLINK Lite，及其配套的USB线缆、转接板、下载线

b) 硬件连接

- 5V电源连接到开发板的电源口。
- 平头哥CKLINK Lite连接到开发板的DEBUG口。

2. 外设使用示例

本章节主要介绍如何使用 API 函数配置芯片外设，达到所需功能。

2.1. UART

UART 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电、串口硬件连线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 example_uart.c 文件中的具体功能示例代码，主要包括：

- a) 使用 CPU 收发数据
- b) 使用 DMA 收发数据
- c) 使用 UART 的超时中断
- d) 使用 IOCTL_CMD 配置 UART 的波特率

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\uart
```

包含以下文件：

```
example_uart.c  
example_uart.h
```

2.1.1. 硬件连接

1) 硬件资源

- 平头哥CKLINK Lite，及其配套的USB线缆、转接板、下载线

- USB转TLL串口模块

2) 硬件连接

- USB转TTL串口模块的TX引脚连接至开发板的PA6，RX引脚连接至开发板的PA5。
- 同时，USB转TTL串口模块连接到电脑。
- 接上开发板电源。

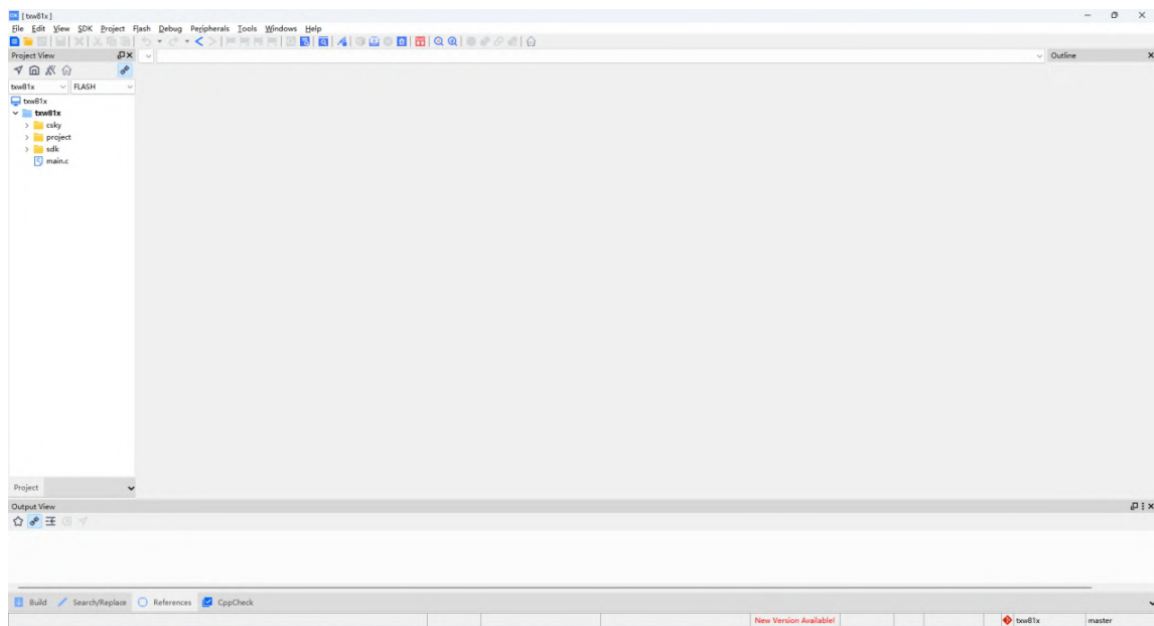
2.1.2. 软件配置

1) 打开工程

打开位于“... \project\txw81x”下的 txw81x.cdkproj，打开工程。如下图：

名称	修改日期	类型	大小
app	2023/11/2 11:49	文件夹	
.git	2023/11/7 11:27	文件夹	
.cdk	2023/11/10 10:49	文件夹	
makecode.exe	2023/10/20 10:01	应用程序	402 KB
crc.exe	2023/11/2 11:49	应用程序	318 KB
BinScript.exe	2023/10/20 10:01	应用程序	327 KB
CSKYFlashProgramerCfg	2023/11/2 11:49	文件	2 KB
txw81x.txt	2023/11/11 11:31	文本文件	8 KB
makecode.ini	2023/11/11 11:30	配置设置	4 KB
txw81x.modify.bat	2023/11/11 11:31	Windows 批处理...	1 KB
sdkrel.bat	2023/11/2 11:49	Windows 批处理...	4 KB
merge.bat	2023/11/2 11:49	Windows 批处理...	1 KB
makedis.bat	2023/11/2 11:49	Windows 批处理...	1 KB
makecode_post.bat	2023/11/2 11:49	Windows 批处理...	1 KB
CSKYFlashProgramerConsole.bat	2023/11/2 11:49	Windows 批处理...	1 KB
precompile.sh	2023/11/2 11:49	SH 源文件	1 KB
prebuild.sh	2023/11/2 11:49	SH 源文件	1 KB
BuildBIN.sh	2023/11/2 11:49	SH 源文件	1 KB
project.map	2023/11/11 11:31	MAP 文件	1,046 KB
txw81x.mk	2023/11/11 11:31	Makefile 源文件	296 KB
cdkws.mk	2023/11/11 11:31	Makefile 源文件	1 KB
project.hex	2023/11/11 11:31	HEX 文件	289 KB
project.elf	2023/11/11 11:31	ELF 文件	1,356 KB
parameter_ui.cfg	2023/11/2 11:49	Configuration 源...	1 KB
parameter.cfg	2023/11/11 11:31	Configuration 源...	3 KB
txw81x.cdkws	2023/11/9 10:29	CDKWS 文件	1 KB
txw81x.cdkproj	2023/11/10 17:22	CDKPROJ 文件	102 KB

成功打开工程后，如下图：



2) 配置UART0 I/O

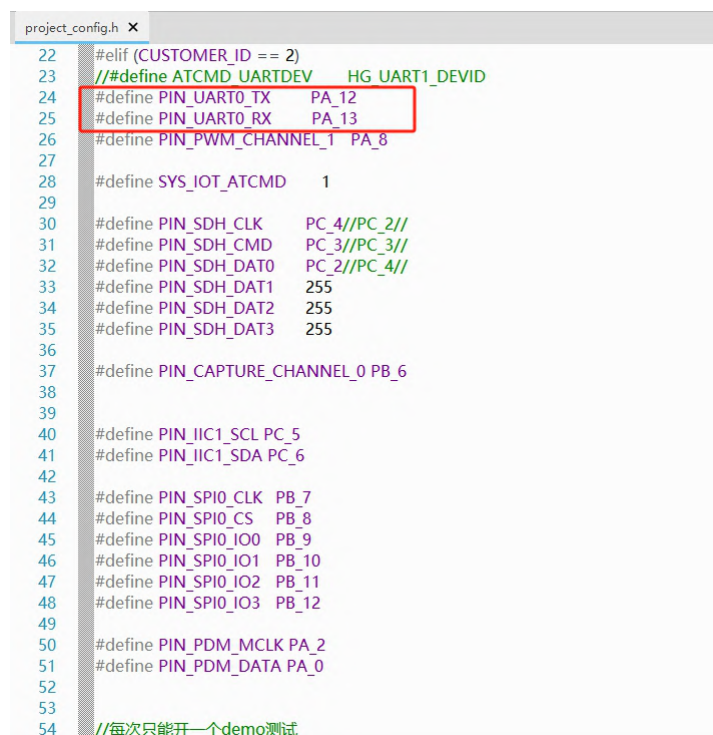
打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 UART0 TX 为 PA_5, RX 为 PA_6 */

#define PIN_UART0_TX PA_5

#define PIN_UART0_RX PA_6
```

完成如下图：



注:

“PIN_UART0_TX”与“PA_5”等有关引脚的宏定义样式, 参加 SDK 文件如下:

```
...\sdk\include\chip\txw81x\pin_names.h
```

该文件定义了各外设的引脚宏定义样式。

2.1.3. 代码示例

以下代码示例的讲解, 内容参考以下文件:

```
...\project\examples\peripherals\uart\example_uart.c
```

```
...\project\examples\peripherals\uart\example_uart.h
```

1) 使用CPU收发数据

- 本例实现的主要功能

电脑上的串口助手发送内容后, UART 原封不动地将内容发回给串口助手

- 关键代码讲解

本例位于 `example_uart_transmit_by_cpu` 函数内, 主要实现思路如下:

UART0 配置 115200 波特率, 申请 UART 接收 1 帧数据完成中断。函数内使用信号量机制, 一直等待接收 1 帧数据完成,

```
uart_open(p_uart, 115200);用于开启 uart 设备并配置波特率
```

```
uart_request_irq(p_uart, (uart_irq_hdl)example_uart_irq_hdl, UART_IRQ_FLAG_RX_BYTE, (uint32)p_uart);
```

用于申请中断, 其中 `example_uart_irq_hdl` 是用户自定义的中断函数, `UART_IRQ_FLAG_RX_BYTE` 意思是接收 1 个 byte 就会触发中断。第四个参数是向中断函数传递的参数。

代码如下:

```

294
295     if (!p_uart) {
296         return RET_ERR;
297     }
298
299     /*
300     * init semaphore.
301     */
302     os_sema_init(&os_sema_rx_byte_done, 0);
303
304     /*
305     * open UART with 115200 baudrate.
306     */
307     uart_open(p_uart, 115200);
308
309     /*
310     * request UART received completely 1 frame data interrupt.
311     */
312     uart_request_irq(p_uart, (uart_irq_hdl)example_uart_irq_hdl, UART_IRQ_FLAG_RX_BYTE, (uint32)p_uart);
313
314     while (1) {
315         /*
316         * wait for "os_sema_rx_byte_done" semaphore in the "example_uart_irq_hdl" interrupt function.
317         */
318         os_sema_down(&os_sema_rx_byte_done, osWaitForever);
319
320         /*
321         * UART send data which has been received.
322         */
323         uart_putc(p_uart, g_rec_buf);
324     }
325 }
326 #endif
327
328
329
330
331 /**
332  * @brief UART example main function

```

申请UART接收1帧完成中断

使用信号量机制进行等待

1 帧数据接收完成后，进入 example_uart_irq_hdl 中断服务函数，保存接收到的数据，同时发送信号量，如下：

```

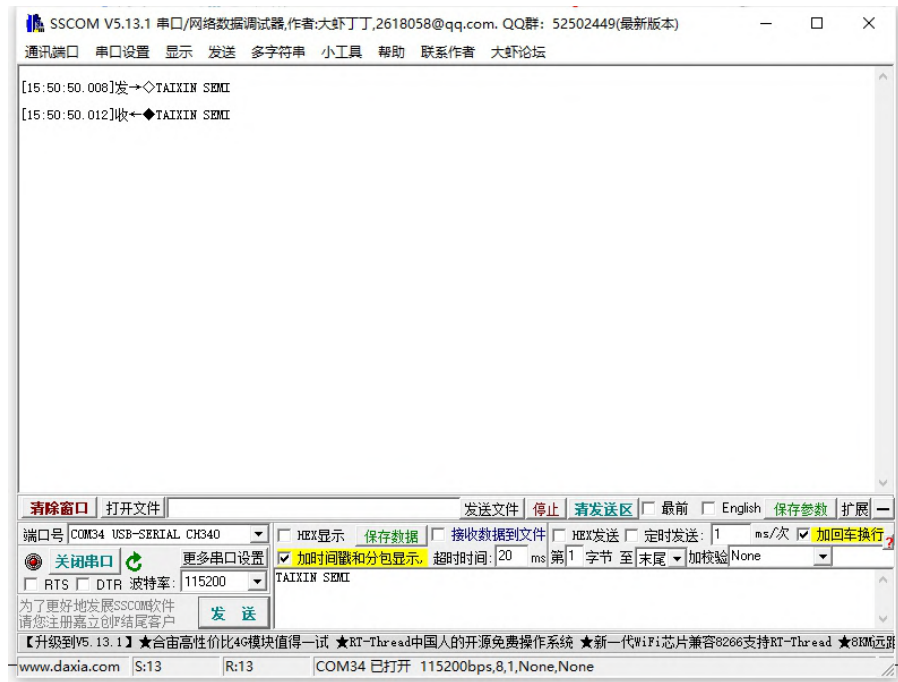
project_config.h  example_uart.c
258 #if EXAMPLE_UART_TRANSMIT_BY_CPU
259
260 /*
261 * semaphore for informing data which has been received successfully.
262 */
263 struct os_semaphore os_sema_rx_byte_done;
264
265 /*
266 * global variable for receiving data.
267 */
268 volatile uint8 g_rec_buf;
269
270 uart_irq_hdl example_uart_irq_hdl(uint32 irq, uint32 irq_data, uint32 param1, uint32 param2)
271 {
272     if (irq == UART_IRQ_FLAG_RX_BYTE) {
273         /*
274         * update the "g_rec_buf" buf to next send.
275         */
276         g_rec_buf = param1;
277
278         /*
279         * send "os_sema_rx_byte_done" semaphore when data received successfully.
280         */
281         os_sema_up(&os_sema_rx_byte_done);
282     }
283
284     return RET_OK;
285 }
286
287 int32 example_uart_transmit_by_cpu(void)
288 {
289     struct uart_device *p_uart = NULL;
290     g_rec_buf = 0;
291
292     p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
293
294     if (!p_uart) {
295         return RET_ERR;
296     }

```

保存接收到的数据

发送信号量

- 下载验证



2) 使用DMA收发数据

- 本例实现的主要功能

电脑上的串口助手发送内容后，UART 原封不动地将内容发回给串口助手

- 关键代码讲解

本例位于 example_uart_transmit_by_dma 函数内，主要实现思路如下：

UART0 配置 115200 波特率，配置使用 DMA，申请 UART DMA 接收完成中断。函数内使用

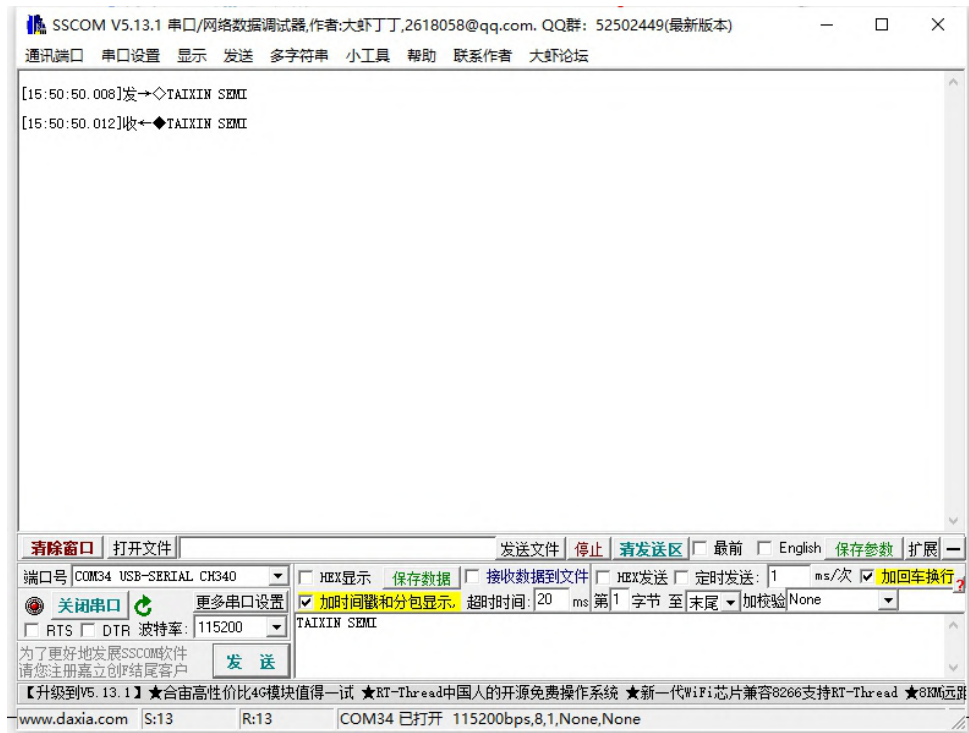
信号量机制，一直等待 DMA 接收数据完成，如下：



DMA 接收数据完成后，进入 example_uart_irq_hdl 中断服务函数，配置下一次 DMA 接收的地址和长度，同时发送信号量，如下：

```
project_config.h example_uart.c main.c
159 * global variable for receiving data.
160 */
161 uint8 g_rec_buf[2][RECEIVE_LENGTH];
162
163 /*
164 * semaphore for informing data which has been received successfully by DMA.
165 */
166 struct os_semaphore os_sema_dma_rx_done;
167
168 uart_irq_hdl example_uart_irq_hdl(uint32 irq, uint32 irq_data, uint32 param1, uint32 param2)
169 {
170     struct uart_device *p_uart = (struct uart_device *)irq_data;
171     if (irq == UART_IRQ_FLAG_DMA_RX_DONE) {
172         g_rbuf_index = (g_rbuf_index + 1) % 2;
173         /*
174          * for next receive, configure UART DMA rx parameter: address and length.
175          */
176         uart_gets(p_uart, g_rec_buf[g_rbuf_index], RECEIVE_LENGTH);
177         /*
178          * send "os_sema_dma_rx_done" semaphore when data received successfully.
179          */
180         os_sema_up(&os_sema_dma_rx_done);
181     }
182     return RET_OK;
183 }
184
185 int32 example_uart_transmit_by_dma(void)
186 {
187     struct uart_device *p_uart = NULL;
188     p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
189     if (!p_uart) {
190         return RET_ERR;
191     }
192 }
```

下载验证



3) 使用UART的超时中断

本例实现的主要功能

配置 UART0 0.1s 的超时中断。电脑上的串口助手连续发送，若发送间隔时间超过 0.1s，

则 UART0 向串口助手发送“T OUT”字样；若发送间隔时间小于 0.1s，则 UART0 不发送任何数据。

- 关键代码讲解

本例位于 example_uart_use_irq 函数内，主要实现思路如下：

UART0 配置 115200 波特率，配置 timeout 时间 0.1s，申请 UART timeout 中断。

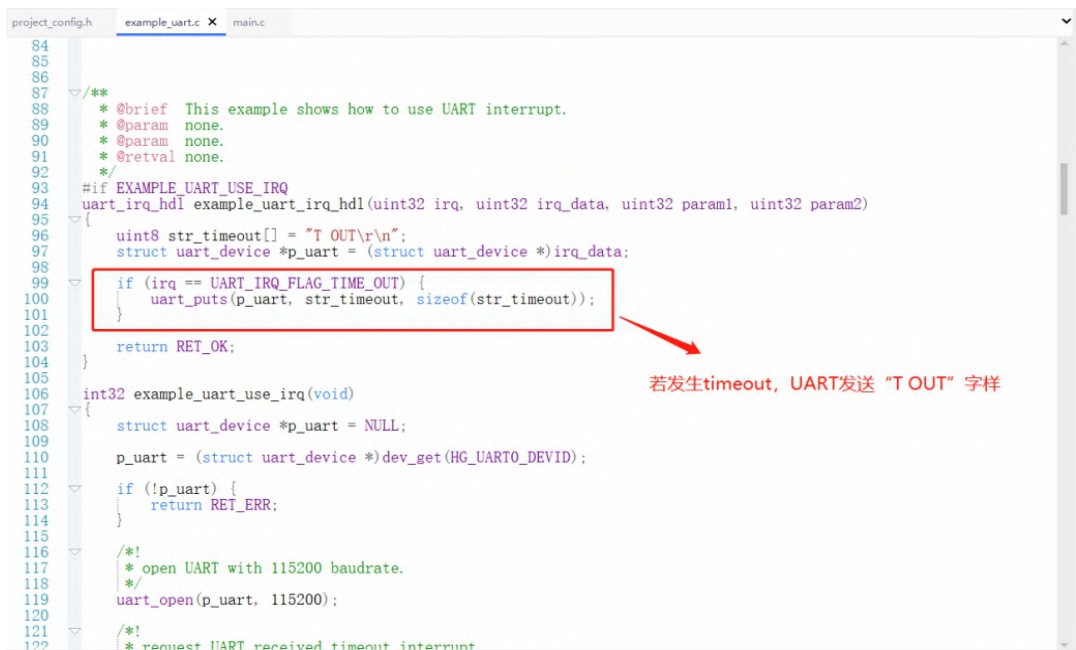


```
102     return RET_OK;
103 }
104
105 int32 example_uart_use_irq(void)
106 {
107     struct uart_device *p_uart = NULL;
108     p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
109     if (!p_uart) {
110         return RET_ERR;
111     }
112     /*
113      * open UART with 115200 baudrate.
114      */
115     uart_open(p_uart, 115200);
116     /*
117      * request UART received timeout interrupt.
118      * note: timeout is valid until received 1 frame data successfully.
119      */
120     uart_request_irq(p_uart, (uart_irq_hdl)example_uart_irq_hdl, UART_IRQ_FLAG_TIME_OUT, (uint32)p_uart);
121     /*
122      * configure UART received timeout, using "uart_ioctl" API.
123      * note: timeout is valid until received 1 frame data successfully.
124      * timeout time formula: 11520 * (1/115200) = 0.1(s)
125      */
126     uart_ioctl(p_uart, UART_IOCTL_CMD_SET_TIME_OUT, 11520, 1);
127     while (1) {
128         os_sleep_ms(10);
129     }
130 }
131 #endif
```

申请UART RX timeout中断

配置UART timeout时间并启动

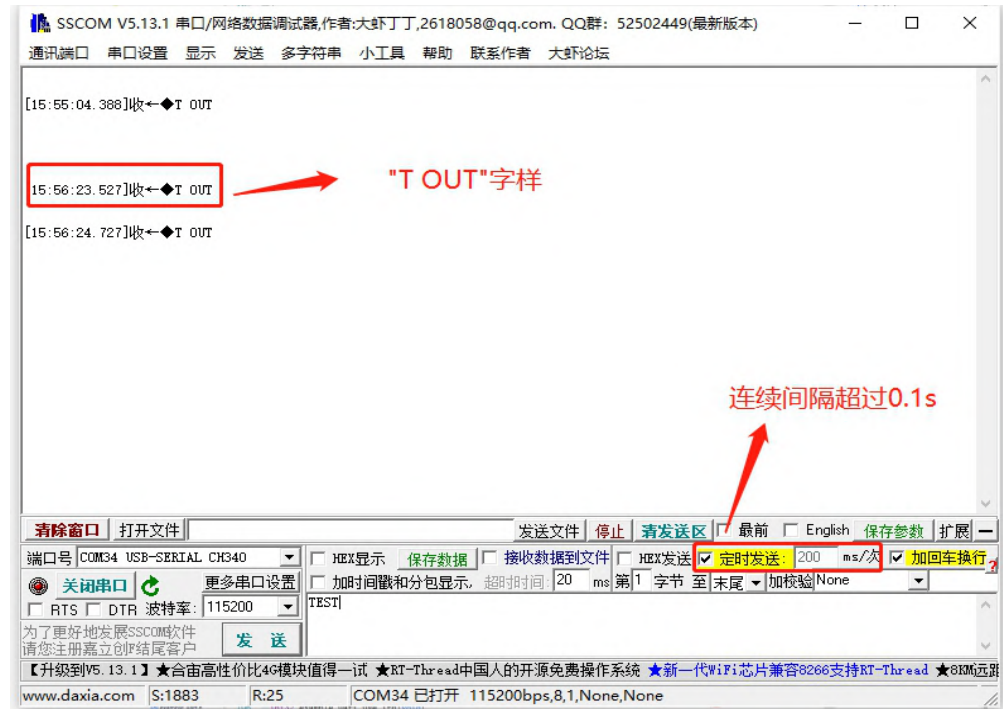
若 UART 发生 timeout 行为，则进入中断，发送“T OUT”字样。



```
84
85
86
87 /**
88  * @brief This example shows how to use UART interrupt.
89  * @param none.
90  * @param none.
91  * @retval none.
92  */
93 #if EXAMPLE_UART_USE_IRQ
94 uart_irq_hdl example_uart_irq_hdl(uint32 irq, uint32 irq_data, uint32 param1, uint32 param2)
95 {
96     uint8 str_timeout[] = "T OUT\r\n";
97     struct uart_device *p_uart = (struct uart_device *)irq_data;
98     if (irq == UART_IRQ_FLAG_TIME_OUT) {
99         uart_puts(p_uart, str_timeout, sizeof(str_timeout));
100     }
101     return RET_OK;
102 }
103
104 int32 example_uart_use_irq(void)
105 {
106     struct uart_device *p_uart = NULL;
107     p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
108     if (!p_uart) {
109         return RET_ERR;
110     }
111     /*
112      * open UART with 115200 baudrate.
113      */
114     uart_open(p_uart, 115200);
115     /*
116      * request UART received timeout interrupt
117      */
```

若发生timeout, UART发送“T OUT”字样

- 下载验证



4) 使用IOCTL_CMD配置UART的波特率

- 本例实现的主要功能

使用 `uart_ioctl` 改变 UART0 的波特率从 115200 到 921600，并以此波特率发送“TAIXIN-SEMI SDK, UART example test”字样到电脑端的串口助手。

- 关键代码讲解

本例位于 `example_uart_use_ioctl_cmd` 函数内，主要实现思路如下：

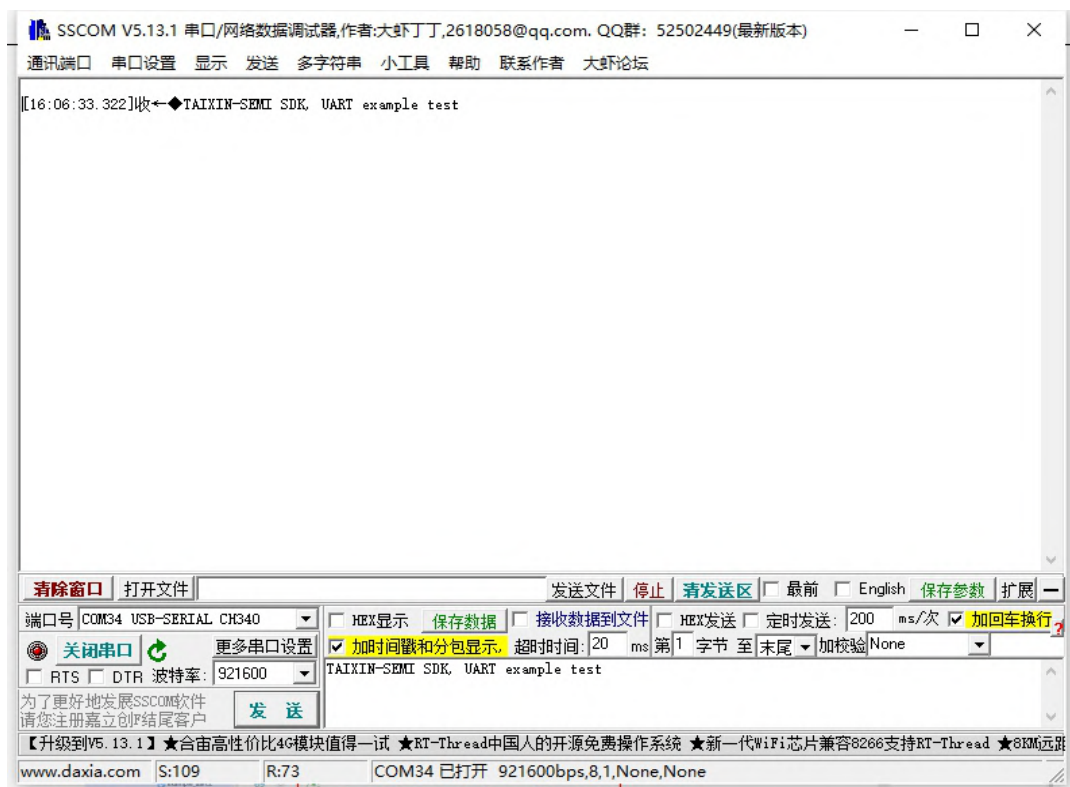
`uart_ioctl(p_uart, UART_IOCTL_CMD_SET_BAUDRATE, 921600, 0);` 用于将波特率设置为 921600。

如下图：

```
project_config.h  example_uart.c  main.c
48  * @param none.
49  * @param none.
50  * @retval none.
51  */
52  #if EXAMPLE_UART_USE_IOCTL_CMD
53  int32 example_uart_use_ioctl_cmd(void)
54  {
55      struct uart_device *p_uart = NULL;
56      uint8 str[] = "TAIXIN-SEMI SDK, UART example test\r\n";
57
58      p_uart = (struct uart_device *)dev_get(HG_UART0_DEVID);
59
60      if (!p_uart) {
61          return RET_ERR;
62      }
63
64      /*
65       * open UART with 115200 baudrate.
66       */
67      uart_open(p_uart, 115200);
68
69      /*
70       * configure UART baudrate, using "uart_ioctl" API.
71       */
72      uart_ioctl(p_uart, UART_IOCTL_CMD_SET_BAUDRATE, 921600, 0);
73
74      uart_puts(p_uart, str, sizeof(str));
75
76      while (1) {
77          os_sleep_ms(10);
78      }
79  }
80  #endif
81
82
83
84
85
86
```

使用uart_ioctl函数改变UART0的波特率

- 下载验证



2.2. PWM

PWM 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电、串口硬件连线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 `example_pwm.c` 文件中的具体功能示例代码，主要包括：

- a) 产生 50% 占空比的 PWM 波形
- b) 使用 PWM 的计数到比较值中断
- c) 使用 `IOCTL_CMD` 改变 PWM 的占空比

示例代码位于 SDK 目录：

```
...\project\examples\peripherals\pwm
```

包含以下文件：

```
example_pwm.c
```

```
example_pwm.h
```

2.2.1. 硬件连接

1) 硬件资源

- USB转TTL串口模块
- 逻辑分析仪

2) 硬件连接

- 逻辑分析仪的采样通道连接到PA8，逻辑分析仪连接至电脑
- USB转TTL串口模块的TX引脚连接至开发板的PA12，RX引脚连接至开发板的PA13。

同时，USB转TTL串口模块连接到电脑。

- 接上开发板电源。

2.2.2. 软件配置

- 1) 打开工程

参考 2.1.2 软件配置

- 2) 配置PWM I/O

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 PA_5 为 PWM CHANNEL 1*/  
  
#define PIN_PWM_CHANNEL_1 PA_1
```

具体细节可参考例 2.1.2UART 的软件配置

注：

“PIN_PWM_CHANNEL_1”与“PA_8”等有关引脚的宏定义样式，参加 SDK 文件如下：

...\sdk\include\chip\txw81x\pin_names.h

该文件定义了各外设的引脚宏定义样式。

2.2.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\pwm\example_pwm.c

...\project\examples\peripherals\pwm\example_pwm.h

- 1) 产生 50%占空比的PWM波形

- 本例实现的主要功能

产生 50%占空比的 PWM 波形，逻辑分析仪捕获波形

- 关键代码讲解

本例位于 `example_pwm_50percent_duty_waveform` 函数内，主要实现思路如下：

PWM 周期值配置 10000，比较值配置 5000 ($10000/2$)，默认使用系统时钟作为计数源。

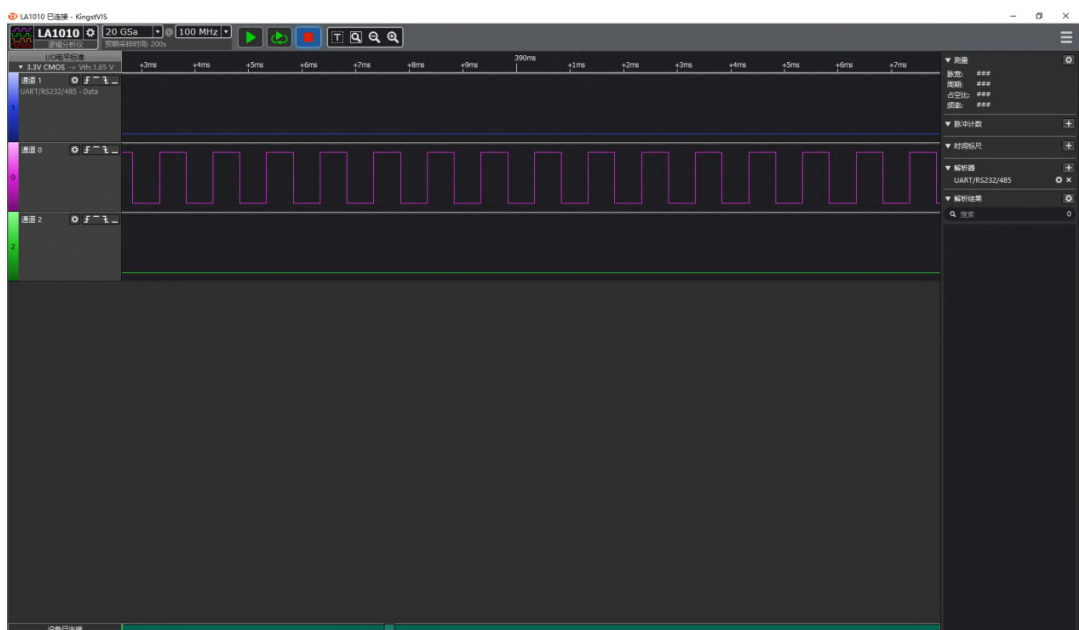
`pwm_init(p_pwm, PWM_CHANNEL_1, 10000, 10000/2);` 用于设置 PWM 的周期值和比较值。

`pwm_start(p_pwm, PWM_CHANNEL_1);` 用于启动 PWM 波发生器

代码如下：

```
123 #endif
124
125
126
127
128
129 /**
130  * @brief This example shows how to generate 50% duty cycle pwm waveform.
131  * @param none.
132  * @param none.
133  * @retval none.
134  */
135 #if EXAMPLE_PWM_50PERCENT_DUTY_WAVEFORM
136 int32 example_pwm_50percent_duty_waveform(void)
137 {
138     struct pwm_device *p_pwm = NULL;
139
140     p_pwm = (struct pwm_device*)dev_get(HG_PWM0_DEVID);
141
142     if (!p_pwm) {
143         return RET_ERR;
144     }
145
146     pwm_init(p_pwm, PWM_CHANNEL_1, 10000, 10000/2); 配置周期值和比较值
147
148     pwm_start(p_pwm, PWM_CHANNEL_1);
149
150     while (1) {
151         os_sleep_ms(10);
152     }
153 }
154 #endif
155
156
157
158
159 /**
160  * @brief PWM example main function.
161  * @param none.
162  * @param none.
```

- 下载验证



2) 使用PWM的计数到比较值中断

- 本例实现的主要功能

开启 PWM 的计数到比较值中断，每次中断到来，UART 打印“INT”字样。

- 关键代码讲解

本例位于 `example_pwm_use_irq` 函数内，主要实现思路如下：

PWM 周期值配置 10000，比较值配置 5000 (10000/2)，默认使用系统时钟作为计数源，

申请 PWM 的计数到比较值中断。

`pwm_request_irq(p_pwm, PWM_CHANNEL_1, PWM_IRQ_FLAG_COMPARE, (pwm_irq_hdl)example_pwm_irq_hdl, 0);` 用于请求比较值中断，当计数值等于比较值时，则会触发中断，进入 `example_pwm_irq_hdl` 函数。

如下：

```
84
85
86 /**
87  * @brief This example shows how to use PWM interrupt.
88  * @param none.
89  * @param none.
90  * @retval none.
91  */
92 #if EXAMPLE_PWM_USE_IRQ
93 pwm_irq_hdl example_pwm_irq_hdl(uint32 irq, uint32 irq_data)
94 {
95     if (irq == PWM_IRQ_FLAG_COMPARE) {
96         os_printf("INT\r\n");
97     }
98     return RET_OK;
99 }
100
101
102 int32 example_pwm_use_irq(void)
103 {
104     struct pwm_device *p_pwm = NULL;
105     p_pwm = (struct pwm_device *)dev_get(HG_PWM0_DEVID);
106     if (!p_pwm) {
107         return RET_ERR;
108     }
109     pwm_init(p_pwm, PWM_CHANNEL_1, 10000, 10000/2);
110     pwm_request_irq(p_pwm, PWM_CHANNEL_1, PWM_IRQ_FLAG_COMPARE, (pwm_irq_hdl)example_pwm_irq_hdl, 0); 使能计数值等于比较值中断
111     pwm_start(p_pwm, PWM_CHANNEL_1);
112     while (1) {
113         os_sleep_ms(10);
114     }
115 }
116 #endif
```

- 下载验证



3) 使用 IOCTL_CMD 改变 PWM 的 占空比

- 本例实现的主要功能

使用 `pwm_ioctl` 函数改变 PWM 的 占空比。

- 关键代码讲解

本例位于 `example_pwm_use_ioctl_cmd` 函数内，主要实现思路如下：

PWM 初始周期值配置 10000，初始比较值配置 5000（10000/2），默认使用系统时钟作

为计数源，每 20ms 使用 `pwm_ioctl` 函数改变 PWM 占空比。

`pwm_ioctl(p_pwm, PWM_CHANNEL_1, PWM_IOCTL_CMD_SET_PERIOD_DUTY, 10000, 10000/2);` 用于重新配置 PWM 的周期值和比较值

如下：

```

42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81

```

```

/**
 * @brief This example shows how to configure PWM module through IOCTL_CMD.
 * @param none.
 * @param none.
 * @retval none.
 */
int32 example_pwm_use_ioctl_cmd(void)
{
    struct pwm_device *p_pwm = NULL;
    uint32 i = 0;

    p_pwm = (struct pwm_device *)dev_get(HG_PWM0_DEVID);

    if (!p_pwm) {
        return RET_ERR;
    }

    pwm_init(p_pwm, PWM_CHANNEL_1, 10000, 10000/2);
    pwm_start(p_pwm, PWM_CHANNEL_1);
    os_sleep_ms(20);

    while (1) {
        if (i%2) {
            pwm_ioctl(p_pwm, PWM_CHANNEL_1, PWM_IOCTL_CMD_SET_PERIOD_DUTY, 10000, 10000/4);
        } else {
            pwm_ioctl(p_pwm, PWM_CHANNEL_1, PWM_IOCTL_CMD_SET_PERIOD_DUTY, 10000, 10000/2);
        }
        i++;
        os_sleep_ms(20);
    }
}

```

每20ms修改一次占空比

2.3. TIMER

TIMER 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电、串口硬件连线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 `example_timer.c` 文件中的具体功能示例代码，主要包括：

a) TIMER 一次计数

b) TIMER 周期计数

c) 使用 `IOCTL_CMD` 改变计数源的分频

示例代码位于 SDK 目录：

```
...\project\examples\peripherals\timer
```

包含以下文件：

```
example_timer.c
```

```
example_timer.h
```

2.3.1. 硬件连接

1) 硬件资源

- USB转TTL串口模块

2) 硬件连接

- USB转TTL串口模块的TX引脚连接至开发板的PA12，RX引脚连接至开发板的PA13。
同时，USB转TTL串口模块连接到电脑。
- 接上开发板电源。

2.3.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置I/O

无

2.3.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\timer\example_timer.c

...\project\examples\peripherals\timer\example_timer.h

1) TIMER一次计数

- 本例实现的主要功能

TIMER 计数一次，计数完成后，打印“once”字样。

- 关键代码讲解

本例位于 example_timer_counts_once 函数内，主要实现思路如下：

TIMER 周期值配置 100000，配置计数类型是单次，默认使用系统时钟作为计数源。

timer_device_open(p_timer, TIMER_TYPE_ONCE, 0);配置定时器类型并打开设备。

timer_device_start(p_timer, 100000, (timer_cb_hdl)example_timer_irq_hdl, 0);

用于设置定时器周期值为 100000，当周期完成时，就会进入 example_timer_irq_hdl 函数。

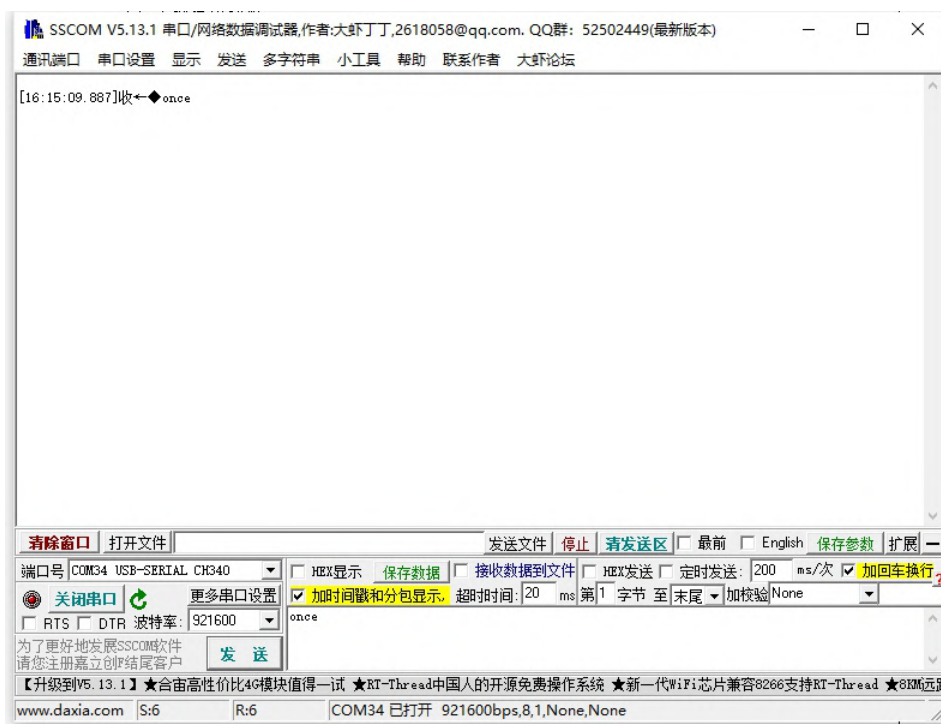
其中第四个参数是中断函数的参数。

代码如下：

```
project_config.h  main.c  example_timer.c X
117
118
119 /**
120  * @brief This example shows how to use TIMER counts once.
121  * @param none.
122  * @param none.
123  * @retval none.
124  */
125 #if EXAMPLE_TIMER_COUNTS_ONCE
126
127 timer_cb_hdl example_timer_irq_hdl(uint32 cb_data, uint32 irq_flag)
128 {
129     os_printf("once\r\n");
130 }
131
132 int32 example_timer_counts_once(void)
133 {
134     struct timer_device *p_timer = NULL;
135
136     p_timer = (struct timer_device*)dev_get(HQ_TIMER0_DEVID);
137
138     if (!p_timer) {
139         return RET_ERR;
140     }
141
142     timer_device_open(p_timer, TIMER_TYPE_ONCE, 0);
143
144     timer_device_start(p_timer, 100000, (timer_cb_hdl)example_timer_irq_hdl, 0);
145
146     while (1) {
147         os_sleep_ms(10);
148     }
149 }
150 #endif
151
152
153
154
155 /**
```

配置TIMER周期计数值

下载验证



2) TIMER周期计数

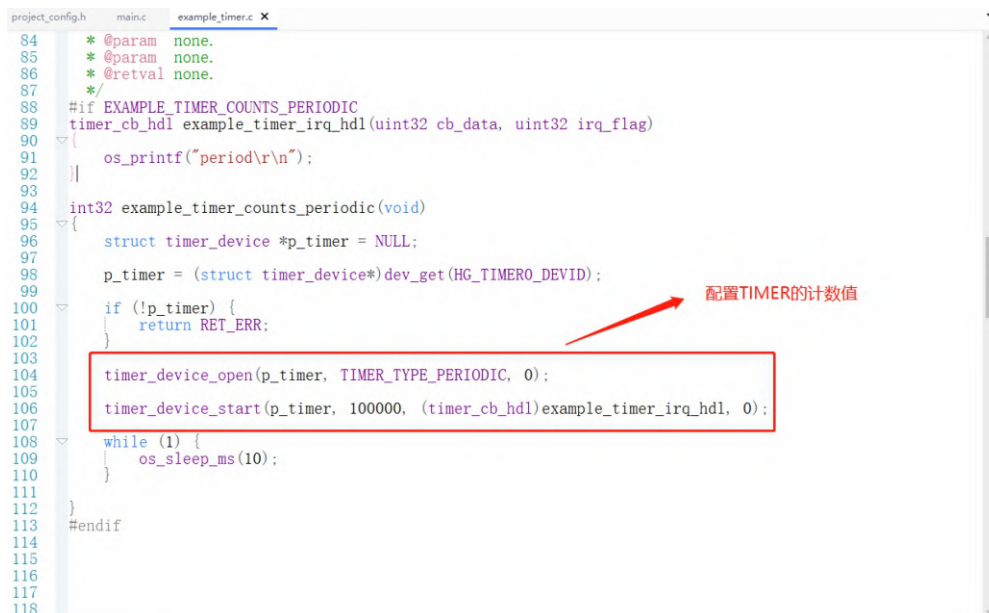
- 本例实现的主要功能

TIMER 周期计数，每次计数完成后，打印“period”字样。

- 关键代码讲解

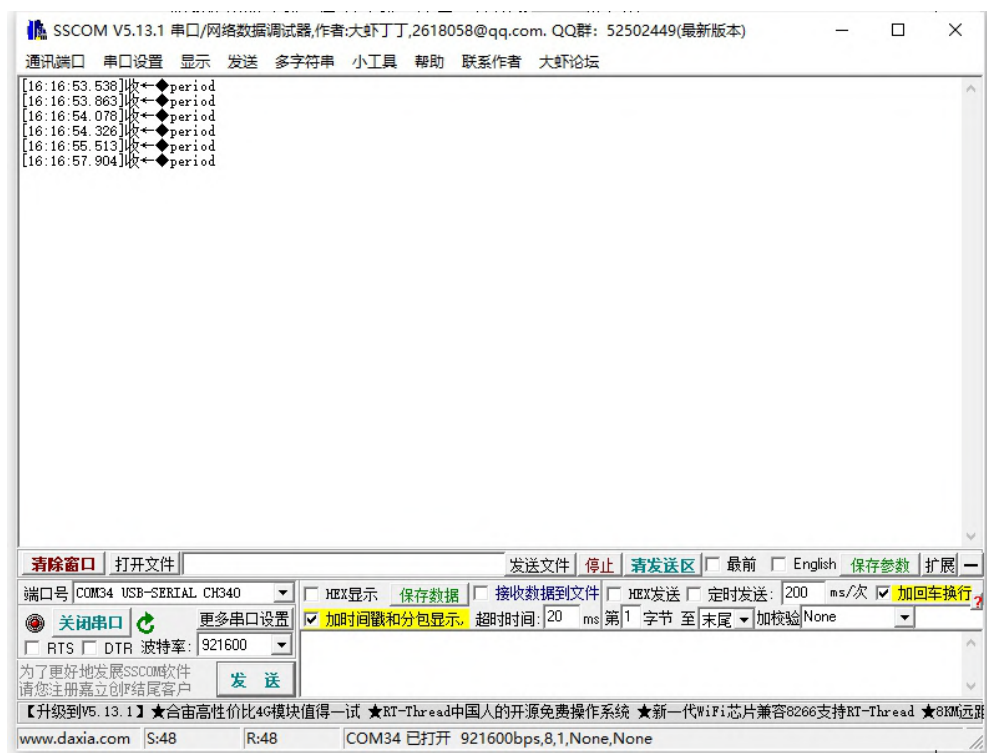
本例位于 example_timer_counts_periodic 函数内，主要实现思路如下：

TIMER 计数值配置 100000，配置周期计数，默认使用系统时钟作为计数源，如下：



```
84  * @param none.
85  * @param none.
86  * @retval none.
87  */
88  #if EXAMPLE_TIMER_COUNTS_PERIODIC
89  timer_cb_hdl example_timer_irq_hdl(uint32 cb_data, uint32 irq_flag)
90  {
91      os_printf("period\r\n");
92  }
93
94  int32 example_timer_counts_periodic(void)
95  {
96      struct timer_device *p_timer = NULL;
97
98      p_timer = (struct timer_device*)dev_get(HG_TIMERQ_DEVID);
99
100     if (!p_timer) {
101         return RET_ERR;
102     }
103
104     timer_device_open(p_timer, TIMER_TYPE_PERIODIC, 0);
105     timer_device_start(p_timer, 100000, (timer_cb_hdl)example_timer_irq_hdl, 0);
106
107     while (1) {
108         os_sleep_ms(10);
109     }
110 }
111
112 #endif
113
114
115
116
117
118
```

- 下载验证



3) 使用 IOCTL_CMD 改变计数源的分频

- 本例实现的主要功能

使用 timer_device_ioctl 函数改变 TIMER 计数源的分频比。每次计数完成后，打印“ioctl cmd”字样。

- 关键代码讲解

本例位于 example_timer_use_ioctl_cmd 函数内，主要实现思路如下：

使用 timer_device_ioctl 函数改变 TIMER 计数源的分频比，从默认 1 分频，改为 32 (2^5) 分频。

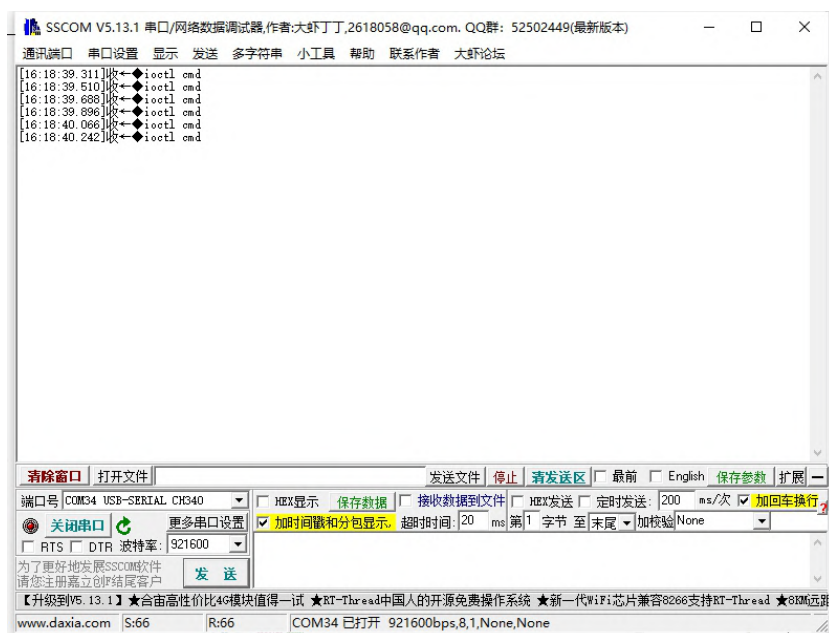
timer_device_ioctl(p_timer, TIMER_SET_CLK_PSC, n, 0); 用于更改定时器分频数，总共有 8 个挡位，第三个参数 n 决定分频数为 2^n 。

如下：



```
54 }
55
56 int32 example_timer_use_ioctl_cmd(void)
57 {
58     struct timer_device *p_timer = NULL;
59
60     p_timer = (struct timer_device*)dev_get(HG_TIMER0_DEVID);
61
62     if (!p_timer) {
63         return RET_ERR;
64     }
65
66     timer_device_open(p_timer, TIMER_TYPE_PERIODIC, 0);
67     timer_device_ioctl(p_timer, TIMER_SET_CLK_PSC, 5, 0);
68     timer_device_start(p_timer, 100000, (timer_cb_hdl)example_timer_irq_hdl, 0);
69
70     while (1) {
71         os_sleep_ms(10);
72     }
73 }
74 #endif
75
76 /**
77  * @brief This example shows how to use TIMER counts periodic.
78  * @param none.
79  * @param none.
80  * @retval none.
81  */
82 #if EXAMPLE_TIMER_COUNTS_PERIODIC
```

- 下载验证



2. 4. I2C

I2C 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电、串口硬件连线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 example_i2c.c 文件中的具体功能示例代码，主要包括：

- a) I2C 发送数据
- b) 使用 I2C 发送所有数据完成中断
- c) 使用 IOCTL_CMD 改变目标 I2C 从机的设备地址

示例代码位于 SDK 目录：

```
...\project\examples\peripherals\i2c
```

包含以下文件：

```
example_i2c.c
```

```
example_i2c.h
```

2. 4. 1. 硬件连接

1) 硬件资源

- USB转TTL串口模块
- 逻辑分析仪

2) 硬件连接

- 逻辑分析仪的采样通道连接到PA5 PA6，逻辑分析仪连接至电脑
- USB转TTL串口模块的TX引脚连接至开发板的PA12，RX引脚连接至开发板的PA13。
- 同时，USB转TTL串口模块连接到电脑。

- 接上开发板电源。

2.4.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置I2C I/O

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* IIC1 */  
  
#define PIN_IIC1_SCL PC_12  
  
#define PIN_IIC1_SDA PC_13
```

具体细节可参考例 2.1.2UART 的软件配置

注：

“PIN_IIC1_SCL”与“PA_5”等有关引脚的宏定义样式，参加 SDK 文件如下：

```
...\sdk\include\chip\txw81x\pin_names.h
```

该文件定义了各外设的引脚宏定义样式。

2.4.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

```
...\project\examples\peripherals\i2c\example_i2c.c
```

```
...\project\examples\peripherals\i2c\example_i2c.h
```

1) I2C发送数据

- 本例实现的主要功能

I2C 发送数据

- 关键代码讲解

本例位于 `example_i2c_transmit` 函数内，主要实现思路如下：

I2C 配置 300kHz 波特率，目标 I2C 设备地址为 0x20，发送 4byte 数据，

`i2c_open(p_i2c, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x20);` 用于配置 iic 主机模式，

7 位地址模式，以及 0x20 的设备地址

`i2c_set_baudrate(p_i2c, 300000);` 用于配置 IIC 的波特率位 300000

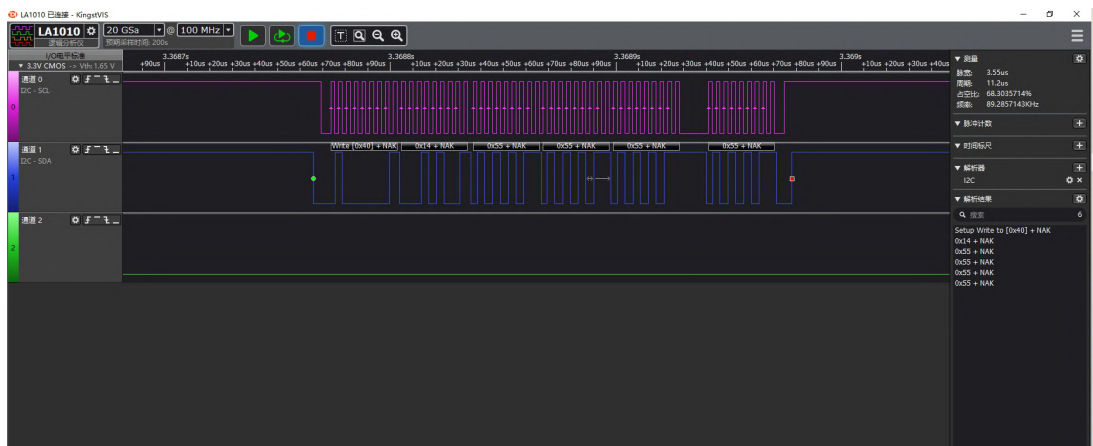
如下：

```
project_config.h  main.c  example_i2c.c x
112 }
113 #endif
114
115
116
117
118 /**
119  * @brief This example shows how to use I2C to transmit.
120  * @param none.
121  * @param none.
122  * @retval none.
123  */
124 #if EXAMPLE_I2C_TRANSMIT
125 int32 example_i2c_transmit(void)
126 {
127     struct i2c_device *p_i2c = NULL;
128     int8 addr_buf[1] = {0x14};
129     int8 data_buf[4] = {0x55, 0x55, 0x55, 0x55};
130
131     p_i2c = (struct p_i2c*)dev_get(HG_I2C1_DEVID);
132
133     if (!p_i2c) {
134         return RET_ERR;
135     }
136
137     i2c_open(p_i2c, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x20);
138     i2c_set_baudrate(p_i2c, 300000);
139
140     i2c_write(p_i2c, addr_buf, 1, data_buf, 4);
141
142     while (1) {
143         os_sleep_ms(1);
144     }
145 }
146 #endif
147
148
149
150
```

配置I2C波特率和从机设备地址

I2C发送4byte数据

- 下载验证



2) 使用I2C发送所有数据完成中断

- 本例实现的主要功能

I2C 发送完 4byte 数据后，进中断打印“I2C Tx Done”字样。

- 关键代码讲解

本例位于 example_i2c_use_irq 函数内，主要实现思路如下：

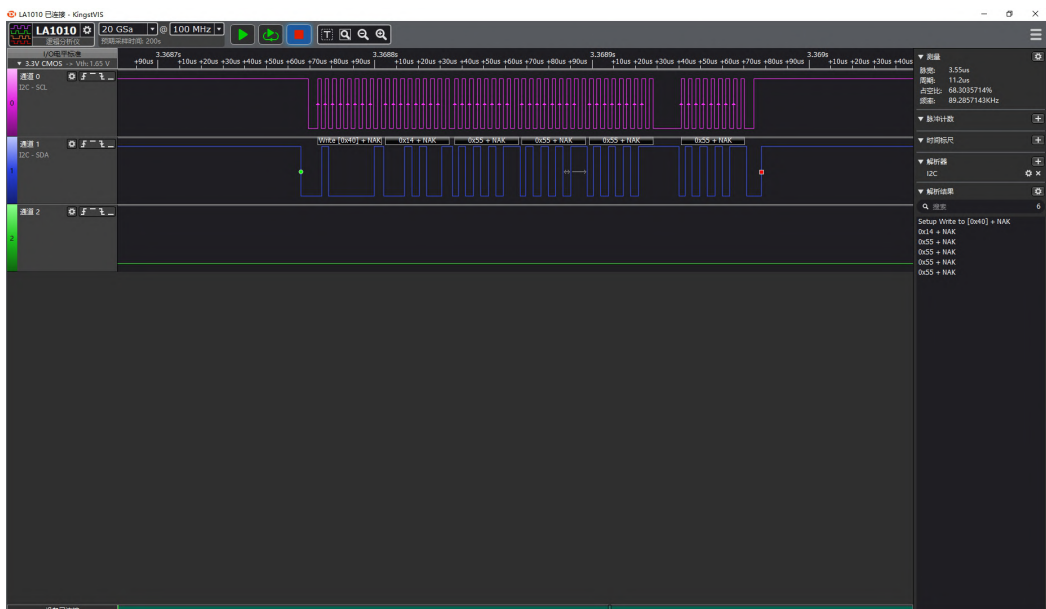
I2C 配置 300kHz 波特率，目标 I2C 设备地址为 0x20，申请发送所有数据完成中断，发送 4byte 数据，如下：

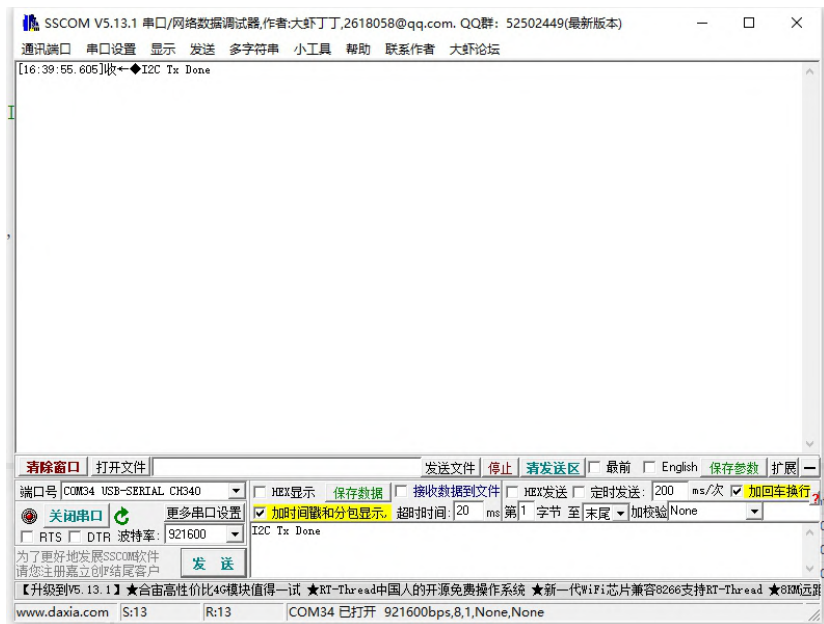
```
project_config.h  main.c  example_i2c.c X
78  * @brief This example shows how to use I2C interrupt.
79  * @param none.
80  * @param none.
81  * @retval none.
82  */
83  #if EXAMPLE_I2C_USE_IRQ
84  i2c_irq_hdl example_i2c_irq_hdl(uint32 irq, uint32 irq_data, uint32 param)
85  {
86      if (irq == I2C_IRQ_FLAG_TX_DONE) {
87          os_printf("I2C Tx Done\r\n");
88      }
89  }
90
91  int32 example_i2c_use_irq(void)
92  {
93      struct i2c_device *p_i2c = NULL;
94      int8 addr_buf[1] = {0x14};
95      int8 data_buf[4] = {0x55, 0x55, 0x55, 0x55};
96
97      p_i2c = (struct i2c_device*)dev_get(HG_I2C1_DEVID);
98
99      if (!p_i2c) {
100          return RET_ERR;
101      }
102
103      i2c_open(p_i2c, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x20);
104      i2c_set_baudrate(p_i2c, 300000);
105
106      i2c_request_irq(p_i2c, (i2c_irq_hdl)example_i2c_irq_hdl, p_i2c, I2C_IRQ_FLAG_TX_DONE);
107
108      i2c_write(p_i2c, addr_buf, 1, data_buf, 4);
109
110      while (1) {
111          os_sleep_ms(1);
112      }
113  }
114  #endif
115
116
```

配置I2C波特率和从机设备地址

申请发送所有数据完成中断

- 下载验证





3) 使用IOCTL_CMD改变目标I2C从机的设备地址

- 本例实现的主要功能

使用 `i2c_ioctl` 函数改变 I2C 从机的设备地址。

- 关键代码讲解

本例位于 `example_i2c_use_ioctl_cmd` 函数内，主要实现思路如下：

I2C 配置 300kHz 波特率，修改目标 I2C 设备地址为 0x30，发送 4byte 数据，

其中 `i2c_ioctl(p_i2c, IIC_SET_DEVICE_ADDR, 0x30)`; 用于将设备地址从 0x20 修改为 0x30。

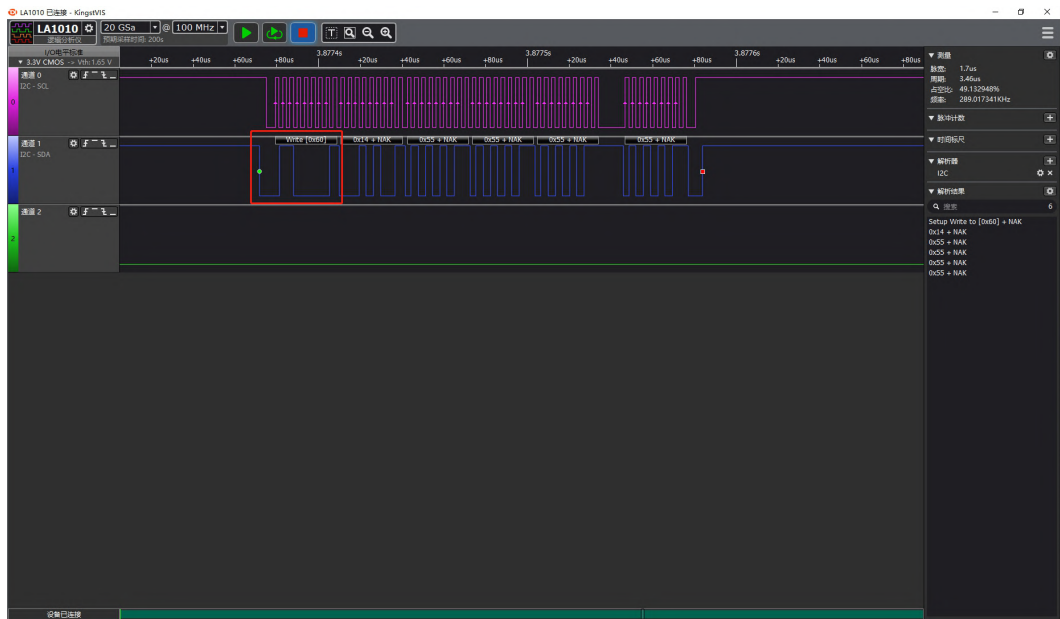
代码如下：

```

39 #define EXAMPLE_I2C_USE_IOCTL_CMD (0)
40
41 /**
42  * @brief This example shows how to configure I2C module through IOCTL_CMD.
43  * @param none.
44  * @param none.
45  * @retval none.
46  */
47 #if EXAMPLE_I2C_USE_IOCTL_CMD
48 int32 example_i2c_use_ioctl_cmd(void)
49 {
50     struct i2c_device *p_i2c = NULL;
51     int8 addr_buf[1] = {0x14};
52     int8 data_buf[4] = {0x55, 0x55, 0x55, 0x55};
53     p_i2c = (struct p_i2c*)dev_get(HG_I2C1_DEVID);
54
55     if (!p_i2c) {
56         return RET_ERR;
57     }
58
59     i2c_open(p_i2c, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x20);
60     i2c_set_baudrate(p_i2c, 300000);
61     i2c_ioctl(p_i2c, IIC_SET_DEVICE_ADDR, 0x30);
62     i2c_write(p_i2c, addr_buf, 1, data_buf, 4);
63
64     while (1) {
65         os_sleep_ms(1);
66     }
67 }
68 #endif
69
70
71
72
73
74
75
76
77

```

- 下载验证



2.5. GPIO

GPIO 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置等。

3) 示例讲解

描述 `example_gpio.c` 文件中的具体功能示例代码，主要包括：

a) GPIO 的输入输出

b) GPIO 的中断

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\gpio
```

包含以下文件：

```
example_gpio.c
```

```
example_gpio.h
```

2.5.1. 硬件连接

1) 硬件资源

- TXW816_WAV_EVB开发板
- 5V电源
- 平头哥CKLINK Lite，及其配套的USB线缆、转接板、下载线

2) 硬件连接

- 5V电源连接到开发板的电源口。
- 平头哥CKLINK Lite连接到开发板的DEBUG口。
- 接上开发板电源。

2.5.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置GPIO I/O

无

2.5.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\gpio\example_gpio.c

...\project\examples\peripherals\gpio\example_gpio.h

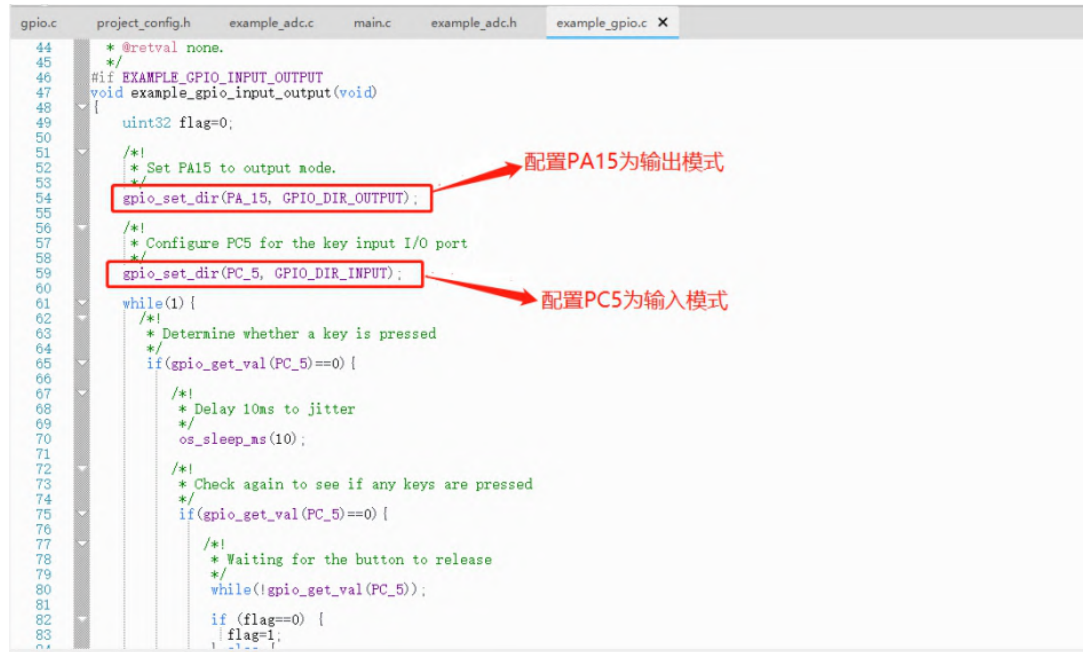
1) GPIO的输入输出

- 本例实现的主要功能
通过按键输入控制 LED 灯的亮灭。
- 关键代码讲解

本例位于 example_gpio_input_output 函数内，主要实现思路如下：

设置 PA15 为输出模式，PC5 为输入模式；通过判断按键是否按下来翻转 LED 灯的亮灭。

如下图所示：



```
44  * @retval none.
45  */
46  #if EXAMPLE_GPIO_INPUT_OUTPUT
47  void example_gpio_input_output(void)
48  {
49      uint32 flag=0;
50
51      /*!
52      * Set PA15 to output mode.
53      */
54      gpio_set_dir(PA_15, GPIO_DIR_OUTPUT);
55
56      /*!
57      * Configure PC5 for the key input I/O port
58      */
59      gpio_set_dir(PC_5, GPIO_DIR_INPUT);
60
61      while(1) {
62          /*!
63          * Determine whether a key is pressed
64          */
65          if(gpio_get_val(PC_5)==0) {
66              /*!
67              * Delay 10ms to jitter
68              */
69              os_sleep_ms(10);
70
71              /*!
72              * Check again to see if any keys are pressed
73              */
74              if(gpio_get_val(PC_5)==0) {
75                  /*!
76                  * Waiting for the button to release
77                  */
78                  while(!gpio_get_val(PC_5));
79
80                  if (flag==0) {
81                      flag=1;
82                  }
83              }
84          }
85      }
```

- 下载验证

依据按键是否按下来控制 LED 灯的亮灭。

2) GPIO的中断

- 本例实现的主要功能

通过按键输入中断，在中断处理函数中控制 LED 灯的亮灭。

- 关键代码讲解

本例位于 example_gpio_use_irq 函数内，主要实现思路如下：

设置 PA15 为输出模式，PC5 为输入模式；通过按键输入，使其进入中断处理函数控制

LED 灯的亮灭。如下图所示：

```
gpio.c  project_config.h  example_adc.c  main.c  example_adc.h  example_gpio.c X
110  gpio_irq_hdl example_gpio_irq_hdl()
111  {
112      /* Flip the IO through the gpio_set_val() function. 中断处理函数
113      */
114      if (g_flag > 0) {
115          gpio_set_val(PA_15, 0);
116          g_flag = 0;
117      } else {
118          gpio_set_val(PA_15, 1);
119          g_flag = 1;
120          os_sleep_ms(20);
121      }
122      return 0;
123  }
124
125  void example_gpio_use_irq()
126  {
127      /* Set PA15 to output mode and do not pull up.
128      */
129      gpio_set_dir(PA_15, GPIO_DIR_OUTPUT);
130      gpio_set_mode(PA_15, GPIO_PULL_UP, GPIO_PULL_LEVEL_NONE); 配置PA15为输出模式不上拉
131
132      /* Configure PC5 as the key input I/O port
133      */
134      gpio_set_dir(PC_5, GPIO_DIR_INPUT); 配置PC为输入模式
135
136      /* Request GPIO PIN interrupt, its IO port and interrupt type
137      */
138      gpio_request_pin_irq(PC_5, (gpio_irq_hdl)example_gpio_irq_hdl, 0, GPIO_IRQ_EVENT_FALL); 申请GPIO下降沿中断
139
140      while (1) {
141          os_sleep_ms(10);
142      }
143  }
144
145  }
```

- 下载验证

依据按键是否按下来控制 LED 灯的亮灭。

2.6. ADC

ADC 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电、串口硬件连线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 example_adc.c 文件中的具体功能示例代码，主要包括：

- a) ADC 采样
- b) ADC 采样完成中断

示例代码位于 SDK 目录：

... \project\examples\peripherals\adc

包含以下文件：

example_adc.c

example_adc.h

2.6.1. 硬件连接

1) 硬件资源

- USB转TTL串口模块

2) 硬件连接

- USB转TTL串口模块的TX引脚连接至开发板的PA12, RX引脚连接至开发板的PA12。同时，USB转TTL串口模块连接到电脑。（如需用其他IO口作为串口使用请参考UART代码示例）
- 接上开发板电源。

2.6.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置GPIO I/O

无

2.6.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\adc\example_adc.c

...\project\examples\peripherals\adc\example_adc.h

1) ADC输入采样

- 本例实现的主要功能

通过串口不断的打印采集到的电压。

- 关键代码讲解

本例位于 example_adc_sample 函数内，主要实现思路如下：

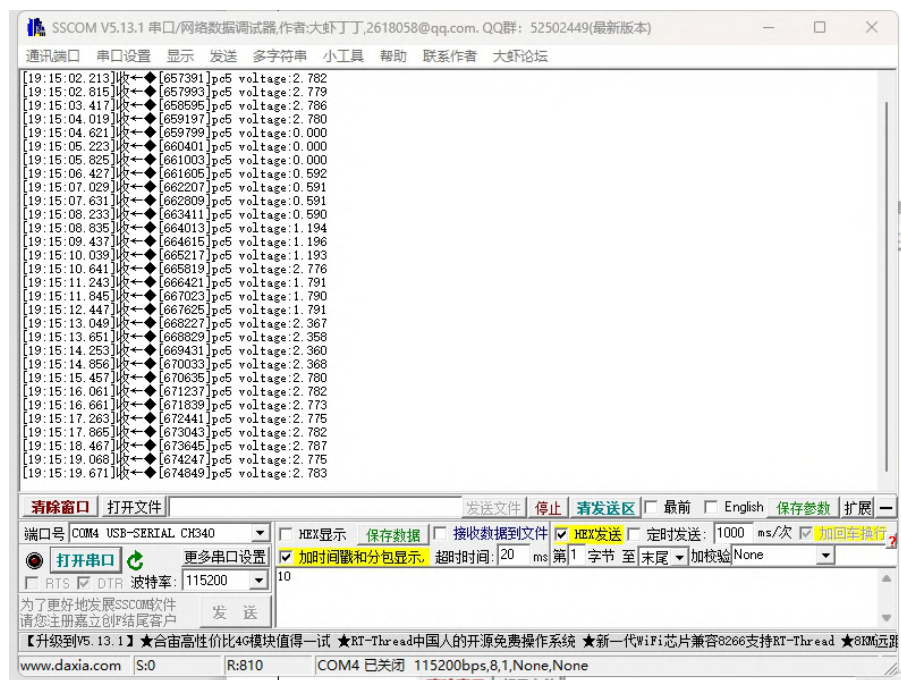
设置 PA3 为 ADC 输入采样通道；通过按下不同的按键采集不同的电压，把采集到的电

压通过串口打印出来。如下图所示：

```
48 #if EXAMPLE_ADC_SAMPLE
49 int32 example_adc_sample(void)
50 {
51     uint32 pa5_voltage=0;
52
53     struct adc_device *p_adc = NULL;
54
55     /*
56      * Get the handle for the ADC via dev_get()
57      */
58     p_adc = (struct adc_device*)dev_get(HG_ADC0_DEVID); 获取ADC句柄
59
60     if (!p_adc) {
61         return RET_ERR;
62     }
63
64     /*
65      * Open ADC
66      */
67     adc_open(p_adc); 打开ADC
68
69     /*
70      * Select ADC Channel
71      */
72     adc_add_channel(p_adc, PA_3); 配置PC5的采样通道
73
74     while(1){
75
76         /*
77          * Obtain the voltage of the PA5 channel
78          */
79         adc_get_value(p_adc, PA_3, &pa5_voltage); 获取ADC采样值
80     }
```

- 下载验证

例程现象如下图所示，当没有按键按下时 ADC 采集到的电压为 2.78V，S1 按下后为 0.00V，S2 按下后为 0.59V，S3 按下后为 1.19V，S4 按下后为 1.79V，S5 按下后为 2.36V。（注意：按键按下后的电压值由 VCC1 和分压比决定，以下数据仅供参考）



2) ADC采样完成中断

- 本例实现的主要功能

在中断处理函数中通过串口不断的打印采集到的电压。

- 关键代码讲解

本例位于 example_adc_use_irq 函数内，主要实现思路如下：

设置 PC5 为 ADC 输入采样通道；通过按下不同的按键采集不同的电压，采集完成后在

中断处理函数中通过串口把采集到的电压打印出来。如下图所示：

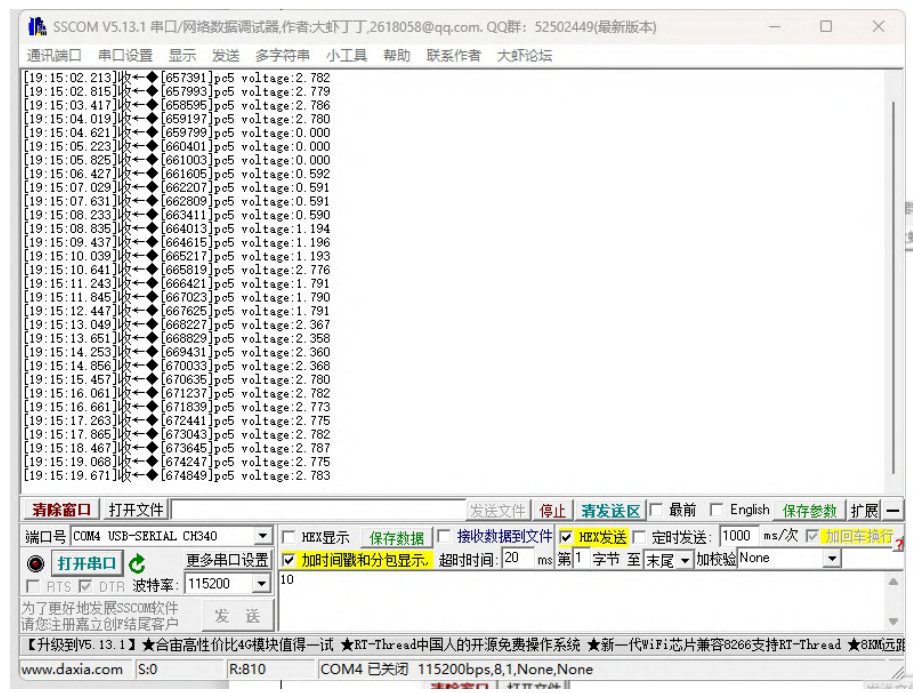
```
101 adc_irq_hdl example_adc_irq_hdl(uint32 irq, uint32 channel, uint32 irq_data)  中断服务函数
102 {
103     if (irq == ADC_IRQ_FLAG_SAMPLE_DONE) {
104         if (channel == PC_5) {
105             os_printf("pa5 voltage:%.3f ", ((double)*irq_data)/4095*3.3);
106         }
107     }
108 }
109
110 int32 example_adc_use_irq()
111 {
112     uint32 pa5_voltage=0;
113     struct adc_device *p_adc = NULL;
114
115     /*
116      * Get the handle for the ADC via dev_get().
117      */
118     p_adc = (struct adc_device*)dev_get(HG_ADC0_DEVID);
119
120     if (p_adc) {
121         return RET_ERR;
122     }
123     /*
124      * Open ADC.
125      */
126     adc_open(p_adc);
127     /*
128      * Select ADC Channel.
129      */
130     adc_add_channel(p_adc, PA_3);
131     /*
132      * Request ADC module sample successfully interrupt.
133      */
134     adc_request_irq(p_adc, ADC_IRQ_FLAG_SAMPLE_DONE, (adc_irq_hdl)example_adc_irq_hdl, 0);  使能采样完成中断，并设置中断服务函数
135     while(1) {
136         /*
137          * Get the voltage from the PC5 channel
138          */
139         adc_get_value(p_adc, PA_3, &pa5_voltage);
140     }
```

- 下载验证

例程现象如下图所示，当没有按键按下时 ADC 采集到的电压为 2.78V，S1 按下后为

0.00V，S2 按下后为 0.59V，S3 按下后为 1.19V，S4 按下后为 1.79V，S5 按下后为 2.36V。（注

意：按键按下后的电压值由 VCC1 和分压比决定，以下数据仅供参考）



2.7. SPI

SPI 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置、I/O 配置等。

3) 示例讲解

描述 example_spi.c 文件中的具体功能示例代码，主要包括：

- a) SPI 通过 DMA 与 W25Q32FV 进行数据读写（读写数据超过 16 字节自动使用 DMA）
- b) SPI 发送完成或接收完成中断

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\spi
```

包含以下文件：

```
example_spi.c
```

```
example_spi.h
```

2.7.1. 硬件连接

1) 硬件资源

- USB转TTL串口模块

2) 硬件连接

- USB转TTL串口模块的TX引脚连接至开发板的PA12，RX引脚连接至开发板的PA13。同时，USB转TTL串口模块连接到电脑。（如需用其他IO口作为串口使用请参考UART代码示例）
- 接上开发板电源。

2.7.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置。

2) 配置SPI I/O

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 SPI I/O */  
  
#define PIN_SPI0_CLK          PB_7  
  
#define PIN_SPI0_CS           PB_8  
  
#define PIN_SPI0_I00          PB_9  
  
#define PIN_SPI0_I01          PB_10
```

具体细节可参考例 2.1.2UART 的软件配置

2.7.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\adc\example_spi.c

...\project\examples\peripherals\adc\example_spi.h

1) SPI通过DMA与W25Q32FV进行数据读写

- 本例实现的主要功能

使用 SPI 的 DMA 模式对 W25Q32FV 进行数据读写, 通过串口打印相关信息

- 关键代码讲解

本例位于 example_spi_transmit 函数内，主要实现思路如下：

设置 SPI 的波特率为 4MHz, 主机模式，线模式为标准模式；上升沿采样，下降沿出数据。根据 W25Q32FV 的相关命令进行操作。如下图所示：

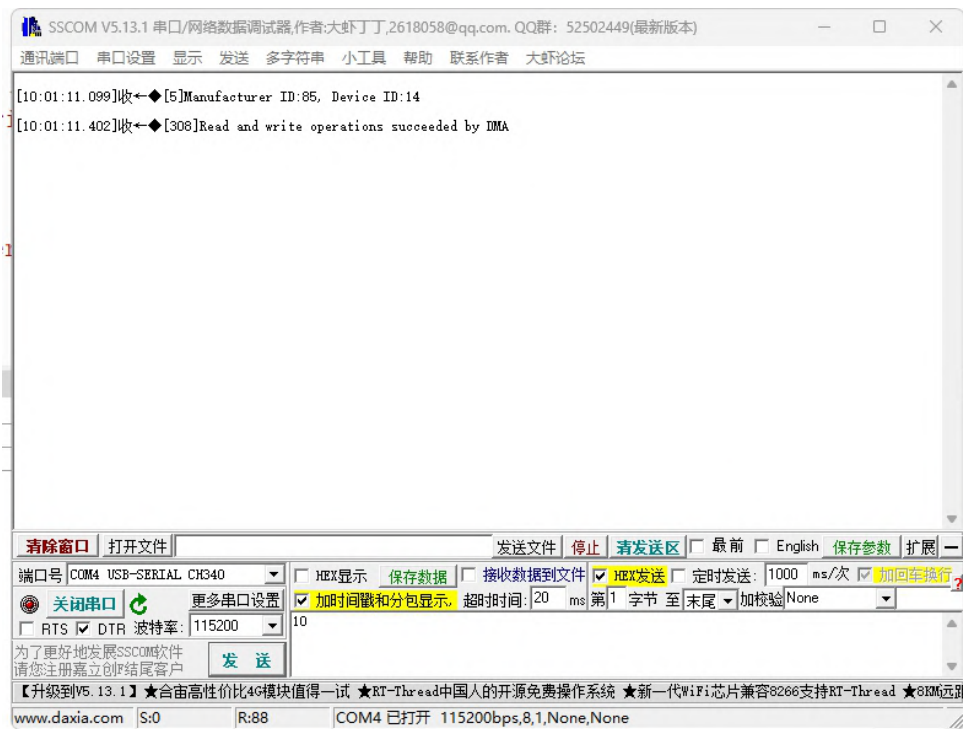
```
gpio.c  project_config.h  example_adc.c  main.c  example_adc.h  example_gpio.c  example_spi.c x
281 int32 example_spi_transmit(void)
282 {
283     struct spi_device *p_spi = NULL;
284     uint8 rx_buf[256] = {0};
285     uint8 tx_buf[256] = {0};
286     uint8 addr_buf[4] = {0};
287     uint16 i = 0;
288
289     /*
290      * Initializes the send and read array.
291      */
292     memset(tx_buf, 0x55, 256);
293     memset(rx_buf, 0x00, 256);
294
295     /*
296      * Get a handle to SPI0.
297      */
298     p_spi = (struct spi_device*)dev_get(HC_SPI0_DEVID);
299
300     /*
301      * Determine whether the SPI handle is obtained correctly.
302      */
303     if (!p_spi) {
304         return RET_ERR;
305     }
306
307     /*
308      * Set the baud rate of SPI to 4000000, host mode, flag line mode,
309      * rising edge sampling and falling edge dividing data.
310      */
311     spi_open(p_spi, 4000000, SPI_MASTER_MODE, SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);
312
313     addr_buf[0] = 0x90;
314     addr_buf[1] = 0x00;
315     addr_buf[2] = 0x00;
316     addr_buf[3] = 0x00;
317
318     /*
319      * Read manufacturer ID and driver ID.
320      */
321     spi_write(p_spi, addr_buf, 4);
322     spi_read(p_spi, rx_buf, 4);
323 }
```

获取SPI句柄

配置SPI的波特率为4MHz, 主机模式, 标准线模式, 上升沿采样, 下降沿出数据

- 下载验证

SPI 通过 DMA 读写示例成功如下图所示。



2) SPI发送和接收完成中断

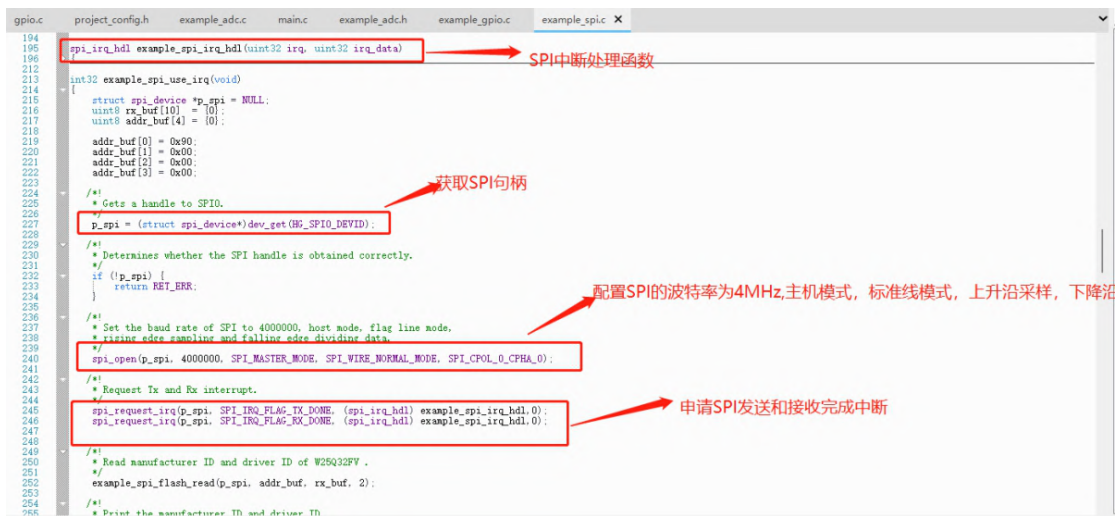
- 本例实现的主要功能

发送或接收完成后在中断处理函数中通过串口打印相关信息。

- 关键代码讲解

本例位于 example_spi_use_irq 函数内, 主要实现思路如下:

设置 SPI 的波特率为 4MHz, 主机模式, 线模式为标准模式; 上升沿采样, 下降沿出数据。根据 W25Q32FV 的相关命令进行操作。如下图所示:



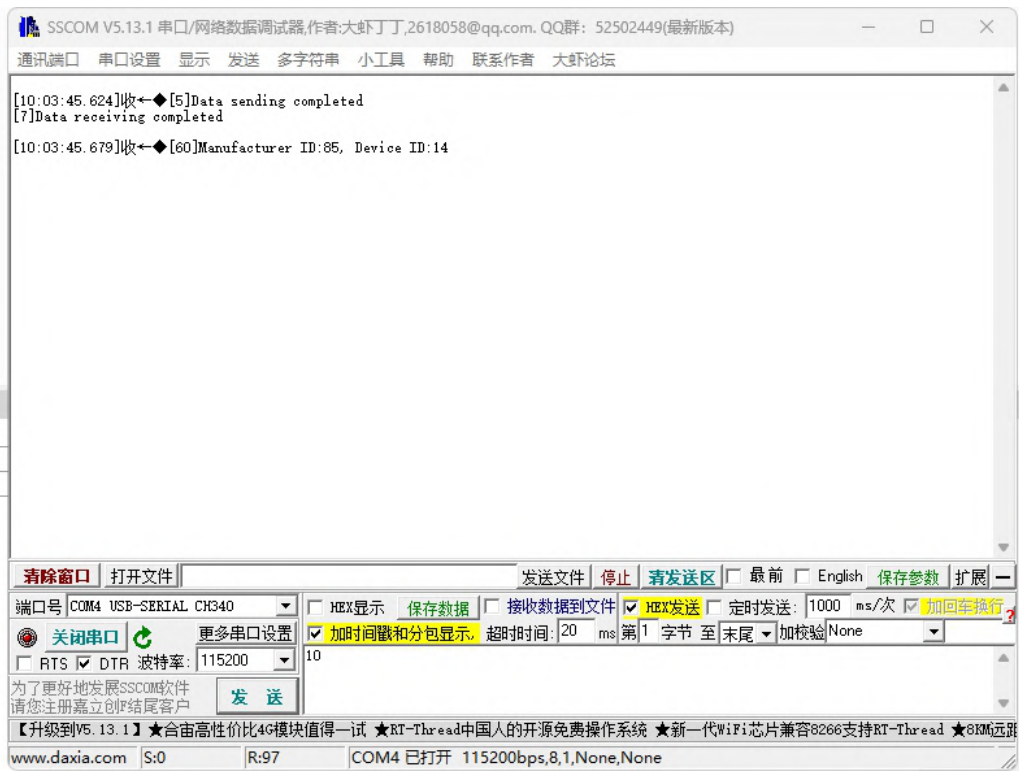
```
194 spi_irq_hdl example_spi_irq_hdl(uint32 irq, uint32 irq_data)
195 {
212
213 }
214
215 int32 example_spi_use_irq(void)
216 {
217     struct spi_device *p_spi = NULL;
218     uint8 rx_buf[10] = {0};
219     uint8 addr_buf[4] = {0};
220
221     addr_buf[0] = 0x90;
222     addr_buf[1] = 0x00;
223     addr_buf[2] = 0x00;
224     addr_buf[3] = 0x00;
225
226     /*
227      * Gets a handle to SPI0.
228      */
229     p_spi = (struct spi_device*)dev_get(HE_SPI0_DEVID);
230
231     /*
232      * Determines whether the SPI handle is obtained correctly.
233      */
234     if (!p_spi) {
235         return RET_ERR;
236     }
237
238     /*
239      * Set the baud rate of SPI to 4000000, host mode, flag line mode,
240      * rising edge sampling and falling edge dividing data.
241      */
242     spi_open(p_spi, 4000000, SPI_MASTER_MODE, SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);
243
244     /*
245      * Request Tx and Rx interrupt.
246      */
247     spi_request_irq(p_spi, SPI_IRQ_FLAG_TX_DONE, (spi_irq_hdl) example_spi_irq_hdl, 0);
248     spi_request_irq(p_spi, SPI_IRQ_FLAG_RX_DONE, (spi_irq_hdl) example_spi_irq_hdl, 0);
249
250     /*
251      * Read manufacturer ID and driver ID of W25Q32FV.
252      */
253     example_spi_flash_read(p_spi, addr_buf, rx_buf, 2);
254
255     /*
256      * Print the manufacturer ID and driver ID.
257     */
258 }
```

Annotations in the image:

- Red arrow pointing to `spi_irq_hdl` function: SPI中断处理函数
- Red arrow pointing to `dev_get` function: 获取SPI句柄
- Red arrow pointing to `spi_open` function: 配置SPI的波特率为4MHz, 主机模式, 标准线模式, 上升沿采样, 下降沿
- Red arrow pointing to `spi_request_irq` functions: 申请SPI发送和接收完成中断

- 下载验证

SPI 发送和接收完成中断示例成功如下图所示。



2.8. CAPTURE

CAPTURE 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电，捕获通道接线等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置等。

3) 示例讲解

描述 `example_capture.c` 文件中的具体功能示例代码，主要包括：

a) CAPTURE 的双沿捕获

b) CAPTURE 的上升沿捕获

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\capture
```

包含以下文件：

```
example_capture.c
```

```
example_capture.h
```

2.8.1. 硬件连接

1) 硬件资源

- USB转TTL串口模块

2) 硬件连接

- PB1 连接PA6
- USB转TTL串口模块的TX引脚连接至开发板的PA12, RX引脚连接至开发板的PA12。同时，USB转TTL串口模块连接到电脑。（如需用其他IO口作为串口使用请参考UART代码示例）
- 接上开发板电源。

2.8.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置CAPTURE I/O

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 SPI I/O */  
  
#define PIN_CAPTURE_CHANNEL_0 PB_6
```

具体细节可参考例 2.1.2UART 的软件配置

2.8.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\capture\example_capture.c

...\project\examples\peripherals\capture\example_capture.h

1) CAPTURE的双沿捕获

- 本例实现的主要功能

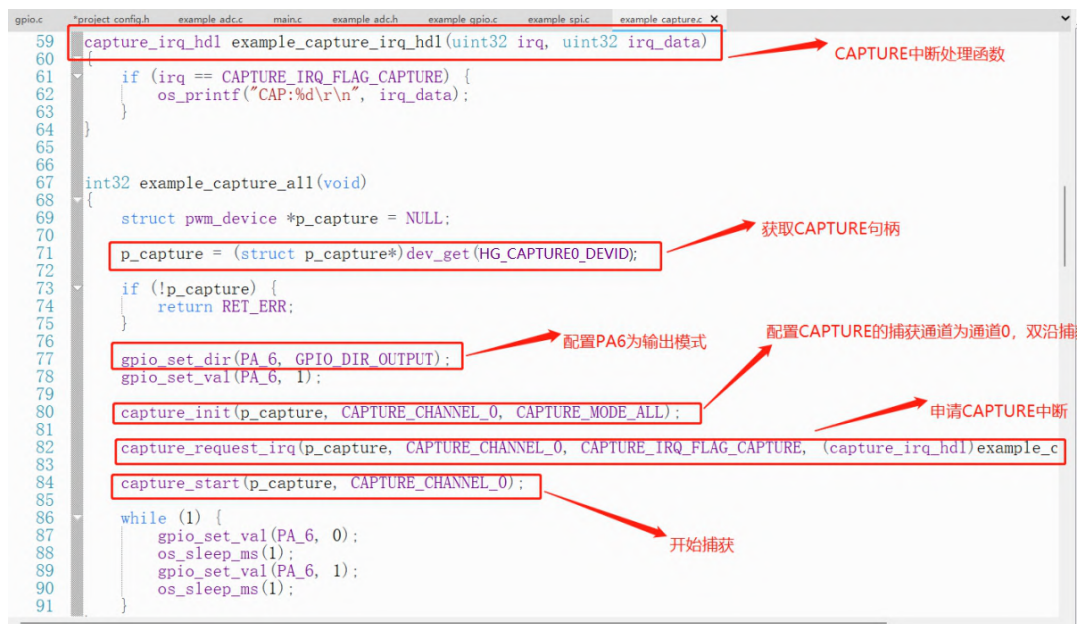
通过串口把捕获到的双沿数据打印出来。

- 关键代码讲解

本例位于 example_capture_all 函数内，主要实现思路如下：

设置 PA6 为输出模式，PB6 为双沿输入捕获模式；捕获 PA6 输出的矩形波。代码如下

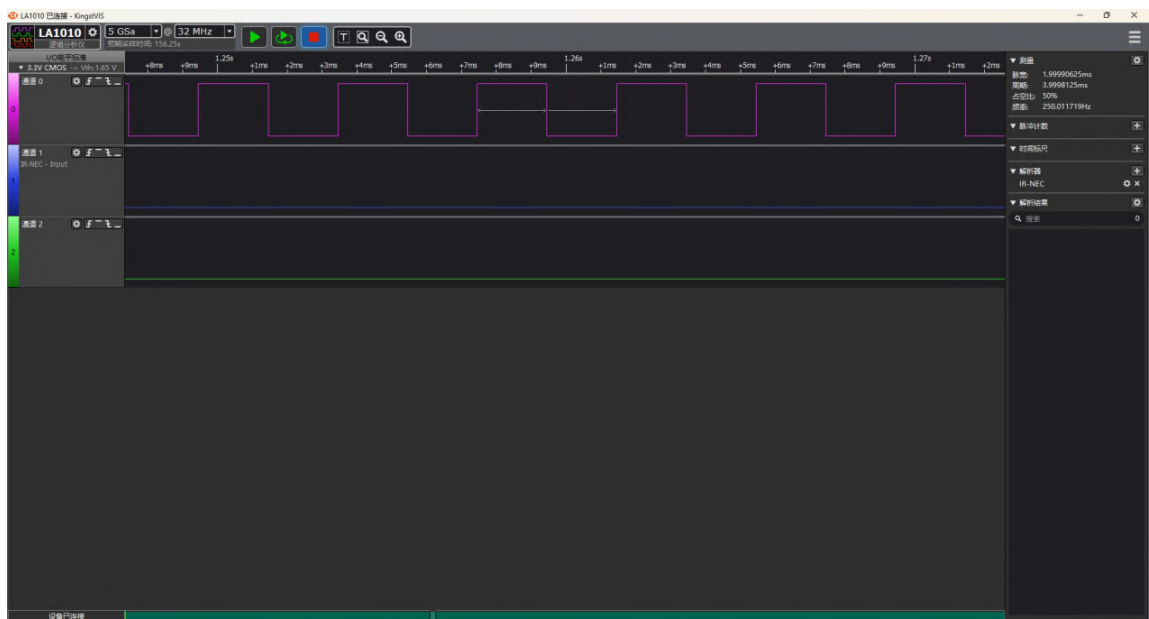
图所示：



- 下载验证

由于打印函数占用了 1ms 的时间且捕获模式为双沿模式，故本次 PA6 输出波形的周期

为 4ms, 占空比 50%。待捕获波形如下图所示：



捕获源的时钟采用系统时钟（180M），理论捕获值为 360000。实际捕获结果如下图所示：



2) APTURE的上升沿捕获

- 本例实现的主要功能

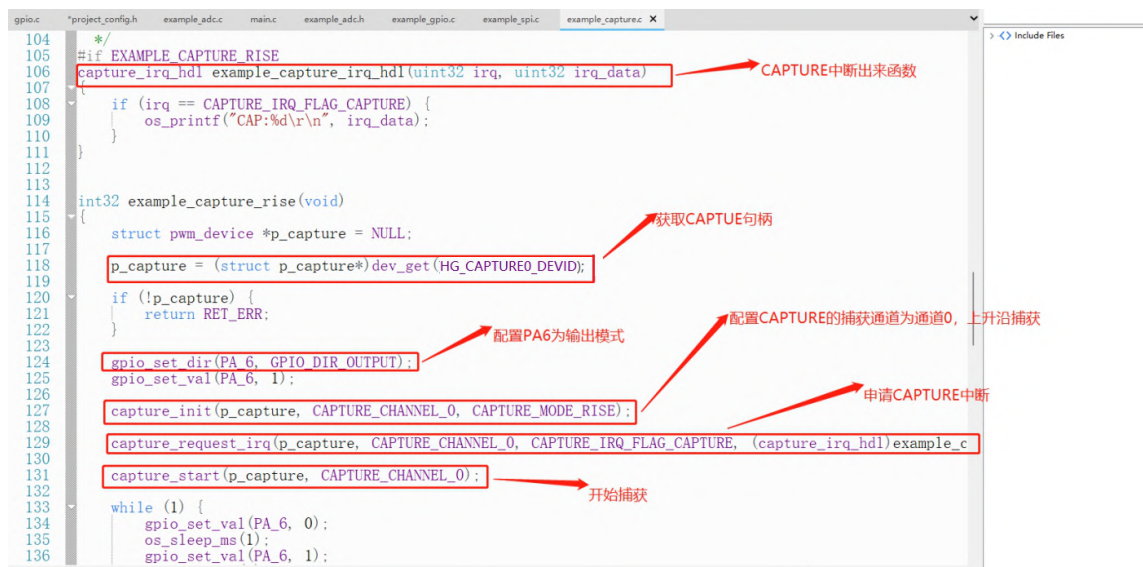
通过串口把捕获到的上升沿数据打印出来。

- 关键代码讲解

本例位于 example_capture_rise 函数内，主要实现思路如下：

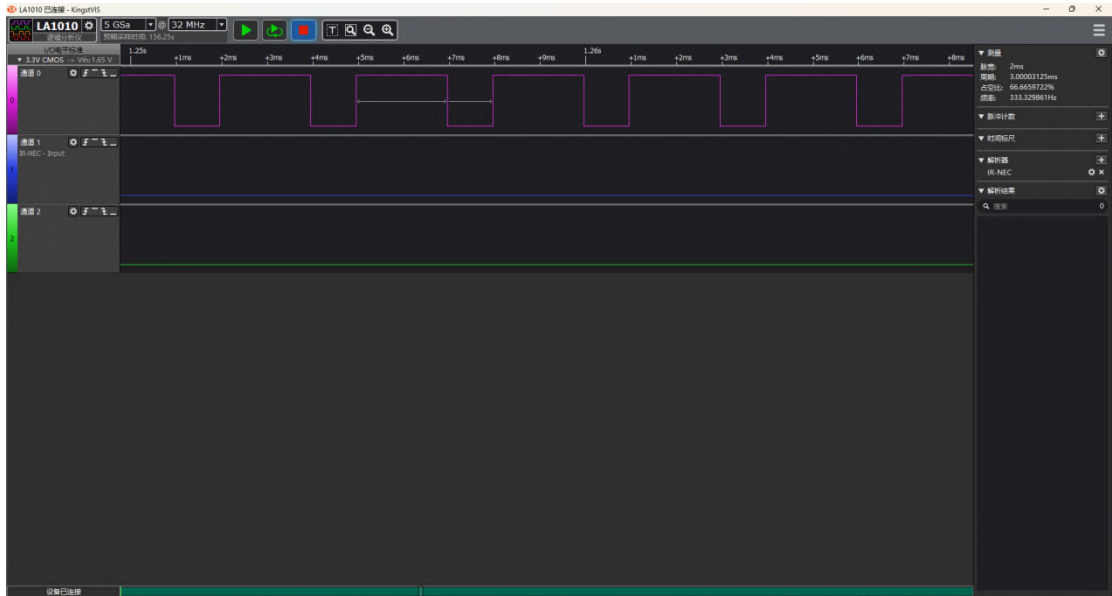
设置 PA6 为输出模式，PB6 为上升沿输入捕获模式；捕获 PA6 输出的矩形波。代码如下

下图所示：



- 下载验证

由于打印函数占用了 1ms 的时间且捕获模式为单沿上升沿模式，故本次 PA6 输出波形的周期为 3ms，占空比 66.666%。待捕获波形如下图所示：



捕获源的时钟采用系统时钟（180M），理论捕获值为 540000。实际捕获结果如下图所示：



2.9. SDHOST

SDHOST 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电，插卡等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置等。

3) 示例讲解

描述 `example_sdhost.c` 文件中的具体功能示例代码，主要包括：

a) 使用 CPU 写读 TF 卡数据

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\sdhost
```

包含以下文件：

```
example_sdhost.c
```

```
example_sdhost.h
```

2.9.1. 硬件连接

1) 硬件资源

- TF存储卡
- USB转TTL串口模块

2) 硬件连接

- 把TF存储卡插在SD卡槽上。
- USB转TTL串口模块的TX引脚连接至开发板的PA12, RX引脚连接至开发板的PA13。同时，USB转TTL串口模块连接到电脑。（如需用其他IO口作为串口使用请参考UART代码示例）。
- 接上开发板电源。

2.9.2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置SDHOST的IO引脚

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 SDHOST 相应引脚 */  
  
#define PIN_SDH_CLK           PC_4  
#define PIN_SDH_CMD           PC_3  
#define PIN_SDH_DAT0          PC_2
```

具体细节可参考例 2.1.2UART 的软件配置

注：

“PIN_SDH_CLK”与“PC_4”等有关引脚的宏定义样式，参加 SDK 文件如下：

...\sdk\include\chip\txw81x\pin_names.h

该文件定义了各外设的引脚宏定义样式。

2.9.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

...\project\examples\peripherals\sdhost\example_sdhost.c

...\project\examples\peripherals\sdhost\example_sdhost.h

1) 使用CPU写读TF卡数据

- 本例实现的主要功能

使用 CPU 实现对 TF 卡的写数据以及读相应区域的数据，查看相应区域的数据是否正确

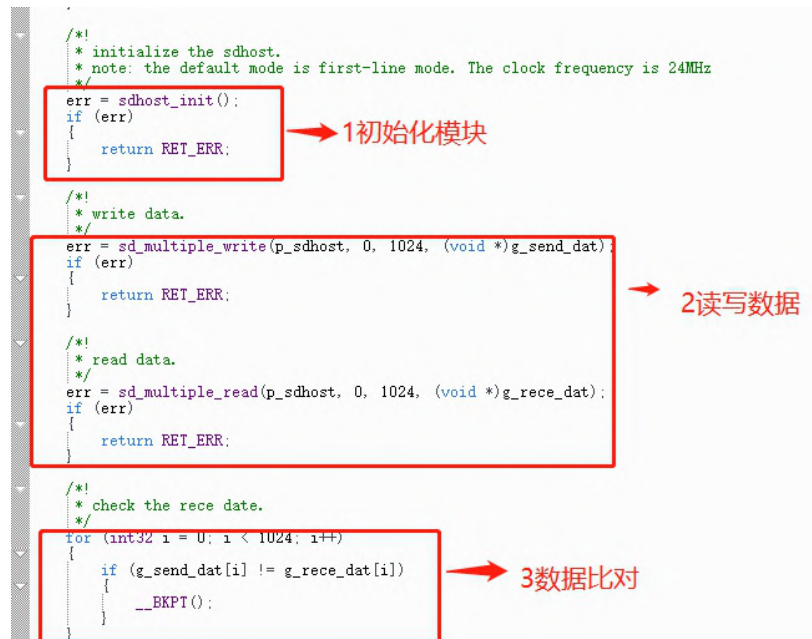
- 关键代码讲解

本例位于 example_sdhost_write_and_read_dat 函数内，主要实现思路如下：

- 打开 SDHOST 设备，然后使用 sdhost_init 函数完成 sdhost 的初始化
- 使用 sd_multiple_write 函数以及 sd_multiple_read 函数完成对 TF 卡的数据读写。
- 最后对数据进行比较判断，查看数据是否正确

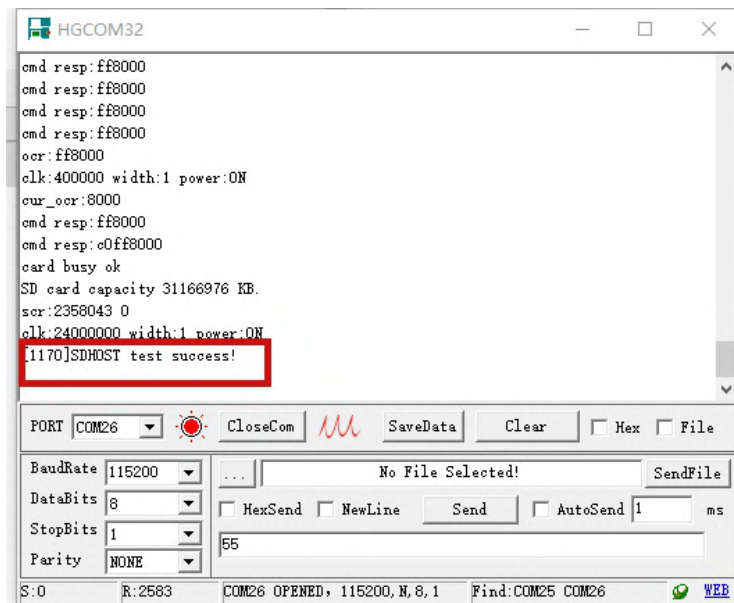
注：

具体如下图所示



- 下载验证

连接串口后可以看到打印“SDHOST test pass”即为成功。



2. 10. PDM_AUDIODAC

PDM_AUDAC 的示例部分，主要包括如下内容：

1) 硬件连接

描述在本示例中，硬件环境如何搭建，主要包括供电，音频子板连接等。

2) 软件配置

描述在本实例中，软件环境如何配置，主要包括集成开发环境的配置等。

3) 示例讲解

描述 example_pdm_audac.c 文件中的具体功能示例代码，主要包括：

a) PDM 与 AUDAC 功能测试

示例代码位于 SDK 目录：

```
... \project\examples\peripherals\pdm_audac
```

包含以下文件：

```
example_pdm_audac.c
```

```
example_pdm_audac.h
```

2. 10. 1. 硬件连接

1) 硬件资源

- 喇叭

2) 硬件连接

- 把喇叭接在开发板的J8卡座上
- 接上开发板电源。

2. 10. 2. 软件配置

1) 打开工程

参考 2.1.2 软件配置

2) 配置PDM、IIS的IO引脚

打开工程左侧的文件目录中“project\project_config.h”，添加如下代码：

```
/* 配置 PDM 相应引脚 */  
  
#define PIN_PDM_MCLK          PA_2  
  
#define PIN_PDM_DATA          PA_0
```

具体细节可参考例 2.1.2UART 的软件配置

注：

“PIN_IIS0_BCLK”与“PA_7”等有关引脚的宏定义样式，参加 SDK 文件如下：

```
...\sdk\include\chip\txw8lx\pin_names.h
```

2.10.3. 代码示例

以下代码示例的讲解，内容参考以下文件：

```
...\project\examples\peripherals\pdm_audac\example_pdm_audac.c
```

```
...\project\examples\peripherals\pdm_audac\example_pdm_audac.h
```

1) PDM与AUDAC功能测试

- 本例实现的主要功能

使用 pdm 模块采集数字麦克风的聲音，然后用芯片内置的音频子系统模块 AUDIO_DAC 输出数字麦克风采集的声音

- 代码思路

本例位于 example_pdm_audac_test 函数内，主要实现思路如下：

- a) 打开 PDM、AUDAC 设备，然后完成两个模块的初始化
- b) 配置 PDM 模块的中断函数，在半满中断中配置新的数据数组，在满中断的时候给出一个信号。

c) 接收到信号时，则用 AUDAC 输出接收完成的数据数组

- 下载验证

下载程序后，对着模块的数字麦克风讲话，聆听喇叭是否听到自己的声音，如果听到就是成功。